逢甲大學 財務金融學系

115 學年度 專題製作

ESG-AI 台股選股平台

程式碼與網站內容對照

（專題「AI 選股結合 ESG 與現有 ETF 的比較」附件，供程式檢查使用）

指導教授：王鈺仁 老師

班　　級：財金系 三年乙班

組　　長：陳東杰 D1280673

組　　員：陳亭翰 D1209072

陳治嘉 D1222280

黃彥霖 D1240092

張建民 D1240121

中華民國 115 年 11 月 1 日

程式碼：GitHub Hendrick0815/ESG_AI_Dashboard　網站：esgaidashboard.streamlit.app

目錄

說明 1

一、系統架構與程式總覽 2

二、各頁共用的設定與運算流程 6

三、首頁（app.py） 10

四、大盤趨勢頁（pages/0_大盤趨勢預測.py） 15

五、策略回測頁（pages/1_策略回測.py） 19

六、個股分析頁（pages/2_個股分析.py） 32

七、模型解釋頁（pages/3_模型解釋.py） 37

八、產業與籌碼頁（pages/4_產業與籌碼.py） 39

九、論文正式版本頁（pages/5_論文正式版本.py） 45

十、選股與回測核心程式（網站與論文共用） 49

十一、資料蒐集、自動更新與部署 55

十二、論文研究程式與輸出檔 59

十三、程式測試 63

說明

本文件說明「ESG-AI 台股選股平台」的程式架構，並將網站每一個頁面對應到產生該畫面的程式段落，使讀者能由畫面追溯計算過程，也能由程式確認網站與論文所呈現的內容。平台以 Python 撰寫，網站以 Streamlit 建置並部署於 Streamlit Community Cloud，程式碼存放於 GitHub（Hendrick0815/ESG_AI_Dashboard）。本文件涵蓋的程式檔共 47 個、約 9,654 行。

各節以對照表列出每項功能所在的程式檔與行數範圍，並簡要說明其作用；行號於產生文件時由程式檔自動定位，與現行程式一致，完整程式碼請見 GitHub 上的對應檔案。截圖取自本機啟動的網站（與雲端網站為同一份程式），截圖時的資料日期以畫面上顯示者為準。

本文件依 2026 年 11 月 1 日的程式產生。專題論文的正式結果使用 2026-10-20 的資料截點，研究設定與網站現行程式一致（每 2 週調倉）；資料截點之後的程式更新（持有期間預估改依大盤狀態、新增短期轉弱減碼選項、大盤狀態特徵開關）於相關章節另外標示，論文結果並未使用這些功能。

對照表之後列出該功能的關鍵程式碼：程式碼由程式檔直接讀取，左側數字為該檔案的實際行號；較長的函式只節錄與計算邏輯相關的部分，省略處以「第 a–b 行略」標示。對照表中的「程式 n」即指本文件中編號相同的程式碼清單。

閱讀時請注意兩項名稱與設定上的差異：

- **策略名稱：**網站程式與畫面中的「單純 ESG」，即論文所稱的「規則式 ESG 篩選投組」；「單純 AI」與「AI+ESG」名稱相同。
- **股票池：**網站「首頁」、「策略回測」、「模型解釋」等頁面以「目前」成交金額前 300 大的股票回測，供日常查詢使用，含有存活者偏誤，報酬會偏高；論文的正式結果以逐期股票池（含之後下市的股票）重新計算，呈現在「論文正式版本」頁與第四章。兩者共用同一套選股與回測程式（第十節），差別只在輸入的股票池與調倉日規則。

一、系統架構與程式總覽

平台分為五個部分：①資料層負責從 Yahoo Finance、證交所、櫃買中心與 TEJ 取得資料並增量更新；②運算層把資料轉為特徵、以 AI 模型滾動訓練、依三種策略選股並回測；③介面層為網站各頁面與共用的圖表元件；④研究層產生論文的正式結果與論文文件；⑤自動化流程每天更新資料，並在程式修改後自動測試與部署。圖 1 為各部分之間的資料流向。



圖 1　系統架構與資料流向

圖 1 呈現平台之系統架構與資料流向。外部資料來源經資料層下載與整理後，交由運算層進行特徵計算、模型訓練與回測，再由介面層呈現於網站；研究層沿用相同之運算程式產生論文之正式結果；自動化流程則負責每日資料更新，以及程式修改後之測試與部署。

表 1　程式檔案清單

程式檔	行數	分類	主要功能	對應的網站內容／論文位置
app.py	150	網站頁面	首頁：資料狀態、最新選股、績效指標卡、累積淨值	首頁

程式檔	行數	分類	主要功能	對應的網站內容／論文位置
pages/0_大盤趨勢預測.py	131	網站頁面	多空指標判斷表、歷史趨勢圖、未來 1 個月展望	大盤趨勢頁
pages/1_策略回測.py	281	網站頁面	績效比較、歷次持股與預估、換股成本、方法說明	策略回測頁
pages/2_個股分析.py	168	網站頁面	個股走勢、風險指標、ESG、同業比較	個股分析頁
pages/3_模型解釋.py	123	網站頁面	特徵重要性、Rank IC、SHAP、訓練紀錄	模型解釋頁
pages/4_產業與籌碼.py	78	網站頁面	產業趨勢、三大法人買賣超	產業與籌碼頁
pages/5_論文正式版本.py	169	網站頁面	論文第三、四章的正式表格與圖	論文正式版本頁；第四章
src/ui/common.py	263	介面元件	側邊欄設定、回測結果快取、持股表與指標卡	所有頁面
src/ui/charts.py	356	介面元件	Plotly 圖表（策略配色固定）	所有圖表
src/_fresh.py	24	介面元件	雲端更新程式後重新載入模組	所有頁面
src/cache.py	99	介面元件	回測結果存檔（資料內容雜湊）	首頁、策略回測、模型解釋
src/config.py	135	核心運算	共用設定：路徑、交易成本、調倉、模型參數	全部；表 3-1、表 3-9
src/pipeline.py	209	核心運算	串接資料→特徵→模型→選股→回測	圖 1-1；首頁、策略回測
src/features.py	209	核心運算	技術面、見頂訊號、產業特徵；依公告日對齊	第三章第二節；表 3-5
src/models.py	144	核心運算	Random Forest／XGBoost 滾動式訓練	第三章第四節；模型解釋頁
src/backtest.py	310	核心運算	調倉日、策略分數、權重、交易成本、回測	第三章第三、四節
src/metrics.py	134	核心運算	年化報酬、波動、Sharpe、回撤、Beta、Alpha	第三章第五節；績效表
src/forecast.py	240	核心運算	持有期間預估路徑與區間	策略回測「歷次持股」
src/market_trend.py	288	核心運算	大盤多空判斷與展望模型	大盤趨勢頁
src/analysis.py	64	核心運算	產業趨勢與法人買賣超彙整	產業與籌碼頁
src/store.py	495	資料層	整合爬蟲與本機檔案、增量更新、股票池、Dataset	首頁資料狀態；所有頁面
src/crawlers/yahoo.py	108	資料層	Yahoo Finance 股價批次下載	股價、比較基準
src/crawlers/twse.py	295	資料層	證交所清單、估值、三大法人、成交金額	估值；產業與籌碼頁
src/loaders/tej.py	87	資料層	讀取 TEJ TESG（公告日為可得日）	ESG 資料；表 3-2
src/loaders/local.py	81	資料層	讀取自備 CSV（示範資料）	合成示範資料
src/utils.py	79	資料層	代號格式、檔案讀寫工具	—
src/research/exchange.py	273	論文研究	逐日行情、價格面板、逐期股票池	第三章第一節；表 3-3

程式檔	行數	分類	主要功能	對應的網站內容／論文位置
src/research/stats.py	83	論文研究	HAC、CAPM、差異檢定、Holm、bootstrap	第三章第五節；第四章第三節
src/research/analysis.py	119	論文研究	市場情境、參與率、年度報酬、ESG 品質	表 4-2、4-3、4-5、4-6
scripts/research/run_research.py	493	論文研究	論文正式結果與 manifest	第四章全部表格；附錄 A
scripts/research/check_twse_coverage.py	72	論文研究	上市股票涵蓋率檢查	第三章第一節（三）
scripts/research/build_report.py	1,085	論文研究	產生論文 Word／PDF	論文全文
scripts/research/report_lib.py	344	論文研究	論文排版工具	全文
scripts/research/appendix_code.py	748	論文研究	論文附錄 A 與本程式對照文件的內容	論文附錄 A；本文件
scripts/research/build_code_doc.py	120	論文研究	產生本程式對照文件（Word）	本文件
scripts/research/capture_web.py	85	論文研究	論文表格截圖	表 3-3～4-14
scripts/research/capture_pages.py	139	論文研究	網站各頁截圖	本文件各圖
scripts/update_data.py	111	資料與自動化腳本	指令列資料更新（每日排程呼叫）	網站資料
scripts/run_backtest.py	60	資料與自動化腳本	指令列回測，預先產生網站存檔	網站回測結果
scripts/auto_push.py	198	資料與自動化腳本	程式變動後自動測試並推送 GitHub	網站部署
scripts/backfill_daily_quotes.py	83	資料與自動化腳本	證交所／櫃買中心逐日全市場行情	論文資料
scripts/fetch_yahoo_all.py	44	資料與自動化腳本	研究用上市股票 Yahoo 日資料	論文資料
scripts/fetch_finmind_twse.py	42	資料與自動化腳本	研究用 FinMind 上市行情（含下市）	論文資料
.github/workflows/daily_update.yml	49	資料與自動化腳本	每日自動更新排程	網站資料
tests/test_core.py	480	測試	核心邏輯測試（前視偏差、成本、指標等）	第十三節
tests/test_research.py	69	測試	研究模組測試	第十三節
tests/smoke_ui.py	237	測試	各頁面冒煙測試	第十三節

註：行數為程式檔目前的總行數（含註解與空白行）。

表 1 列示本文件涵蓋之全部程式檔及其行數與功能，並依網站頁面、介面元件、核心運算、資料層、論文研究、資料與自動化腳本及測試分類；最右欄為各程式對應之網站內容或論文章節。

表 2　網站頁面與程式對照

網站頁面	頁面程式	主要呼叫的模組與函式	畫面內容
首頁	app.py	pipeline.run()、metrics.performance_table()、charts.nav_chart()、bt.next_rebalance()	資料狀態、最新選股、三策略與基準的年化報酬、累積淨值
大盤趨勢	pages/0_大盤趨勢預測.py	market_trend.current_trend()、trend_series()、run()	7 項指標多空判斷表、歷史趨勢底色圖、未來 1 個月展望
策略回測	pages/1_策略回測.py	pipeline.run()、metrics.performance_table()、forecast.holding_forecast()、backtest.trading_cost()	績效比較、歷次持股與持有期間預估、換股與成本、方法說明
個股分析	pages/2_個股分析.py	crawlers.yahoo.fetch_prices()、metrics.price_metrics()、performance()	個股走勢、風險指標、ESG 評等、同業比較
模型解釋	pages/3_模型解釋.py	pipeline.rank_ic()、models.walk_forward() 的訓練紀錄	Rank IC、特徵重要性、SHAP、訓練紀錄
產業與籌碼	pages/4_產業與籌碼.py	analysis.industry_trend()、flow_leaders()、flow_by_industry()	產業趨勢、三大法人買賣超排行
論文正式版本	pages/5_論文正式版本.py	web_tables.json（build_report.py 產生）、charts.nav_chart()	論文第三、四章的正式表格與圖

表 2 彙整網站七個頁面之對應程式檔、主要呼叫之核心函式及畫面內容，各頁之詳細對照另見第三至九節。

二、各頁共用的設定與運算流程

每個頁面開頭都依相同順序執行：先呼叫 ensure_fresh() 確保雲端更新程式後讀到新版模組，再以 setup_page() 設定版面，接著由側邊欄 sidebar_data() 選擇資料來源、sidebar_strategy() 設定策略參數，最後以 get_result() 取得回測結果。get_result() 依序查詢記憶體、回測存檔（outputs/cache）與實際計算，同一組資料與參數只會計算一次；存檔的鍵值包含資料檔內容的雜湊值與參數，資料每日更新後會自動重新計算。

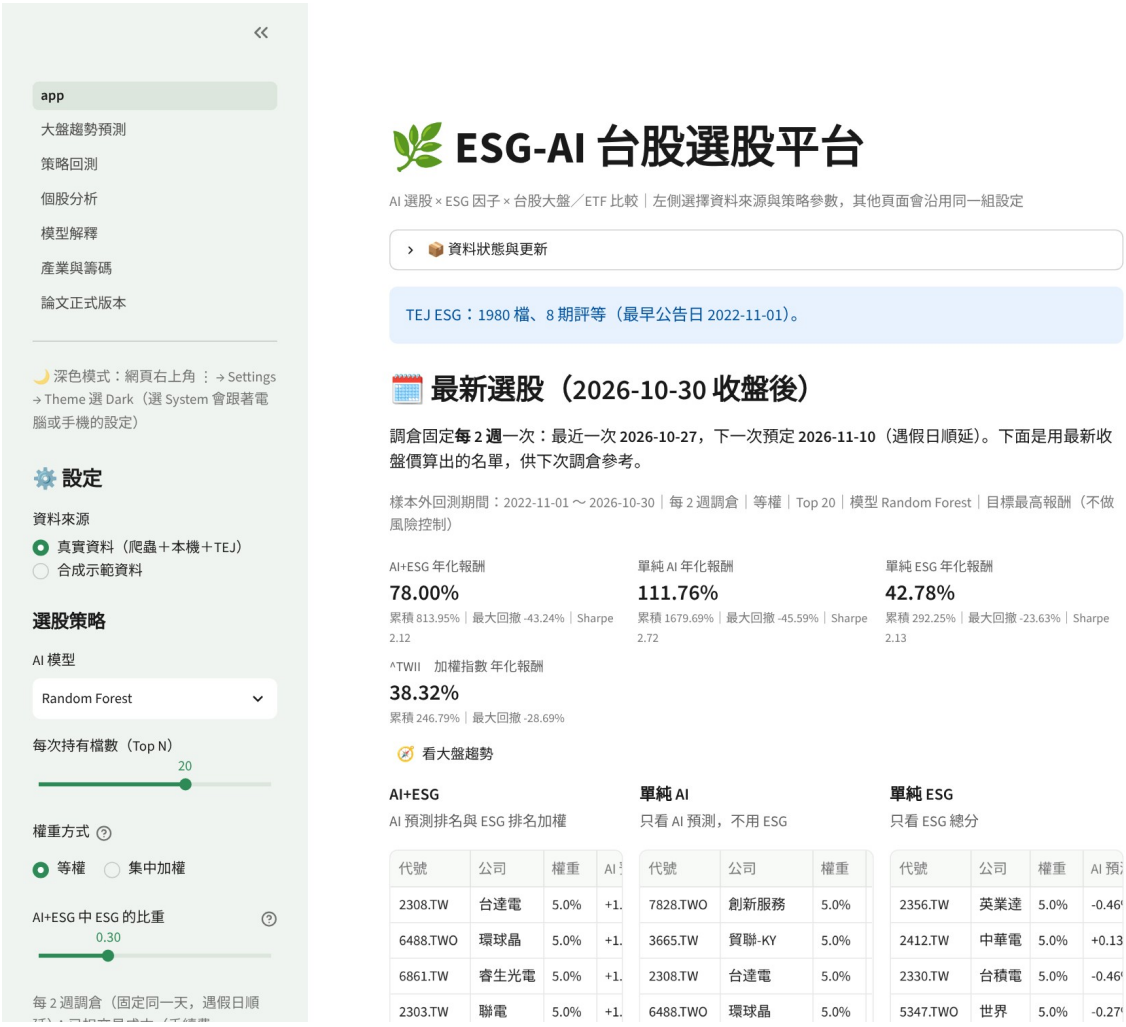


圖 2　側邊欄設定（資料來源與選股策略參數）

圖 2 呈現各頁共用之側邊欄設定區，上方為資料來源選擇，下方為 AI 模型、持股檔數、權重方式、ESG 比重，以及風險控制與短期轉弱減碼等選用功能之設定。各頁面共用同一組設定，任一設定變更均會重新執行回測。

程式 1　src/ui/common.py：側邊欄策略參數 sidebar_strategy()（第 93 – 133 行）

```
def sidebar_strategy(ds: store.Dataset) -> Params:
    models = available_models()
    if not models:
        st.error("請先安裝 scikit-learn：pip install scikit-learn")
        st.stop()
    with st.sidebar:
        st.subheader("選股策略")
        model = _keep("model", st.selectbox("AI 模型", models,
index=models.index(_remember("model", models[0])) if _remember("model", models[0]) in models else
0))
        top_n = _keep("top_n", st.slider("每次持有檔數（Top N）", 3, 30, _remember("top_n",
config.DEFAULT_TOP_N)))
        w_opts = ["等權", "集中加權"]
        weighting = _keep("weighting", st.radio("權重方式", w_opts, horizontal=True,
                                                index=w_opts.index(_remember("weighting",
config.DEFAULT_WEIGHTING)),
                                                help="集中加權：第 1 名 30%，前 5 名約 69%（原
generate_backtest.py 的做法）"))
        esg_w = _keep("esg_w", st.slider("AI+ESG 中 ESG 的比重", 0.0, 1.0, _remember("esg_w",
config.DEFAULT_ESG_WEIGHT), 0.05,
                                         help="混合分數 =（1−比重）× AI 預測排名 ＋ 比重 × ESG 排
名"))
        st.caption(f"每 {config.REBALANCE_WEEKS} 週調倉（固定同一天，遇假日順延）；已扣交易成本（手續
費 0.1425%、賣出證交稅 0.3%）")
        risk_on = _keep("risk_on", st.checkbox(
            "風險控制（降低最大回撤，報酬會變低）", _remember("risk_on",
config.DEFAULT_RISK_CONTROL),
            help="① 不買跌破年線（200 日均線）的股票\n\n"
                 "② 不買近 60 日波動最高的 20% 股票\n\n"
                 "③ 加權指數跌破年線時持股減半，站回年線恢復滿倉\n\n"
                 "預設關閉（以最高報酬為目標）；勾選可以比較降低回撤後的結果。"))
        pull_on = _keep("pull_on", st.checkbox(
            "短期轉弱減碼（大盤回檔時持股減半）", _remember("pull_on",
config.DEFAULT_PULLBACK_CONTROL),
            help=f"加權指數收盤價距近 {config.PULLBACK_WINDOW} 日最高收盤價跌幅超過 {-
config.PULLBACK_DD:.0%} 時，"
                 f"持股降為 {config.PULLBACK_EXPOSURE:.0%}（其餘為現金）；回到 {-
config.PULLBACK_DD:.0%} 以內恢復滿倉。\n\n"
                 "反應比「跌破年線」快，可減少回檔時的虧損與最大回撤；代價是報酬降低、換倉次數增加。"))
        with st.expander("進階"):
            trees = _keep("trees", st.slider("樹的數量", 50, 500, _remember("trees",
config.DEFAULT_N_ESTIMATORS), 50))
            static_esg = _keep("static_esg", st.checkbox(
                "ESG 歷史不足時，用最新一期回填（有前視偏差，僅供對照）", _remember("static_esg",
False)))
            tickers = sorted(ds.prices["ticker"].unique()) if not ds.prices.empty else []
            names = ds.names
            picked = st.multiselect("限定股票池（空白 = 全部）", tickers, default=[t for t in
_remember("pool", []) if t in tickers],
                                    format_func=lambda t: label(t, names))
            _keep("pool", picked)
    return Params(model_name=model, top_n=top_n, n_estimators=trees, esg_weight=esg_w,
                  weighting=weighting, static_esg=static_esg,
                  tickers=tuple(sorted(picked)) or None,
                  risk_control=risk_on, pullback_control=pull_on)
```

程式 2　src/ui/common.py：取得回測結果 get_result()（第 136 – 165 行）

```
def get_result(mode: str, demo_set: Optional[str], params: Params) -> Optional[Result]:
    """順序：記憶體 → outputs/cache 存檔 → 真的計算（顯示進度條，算完存檔）。"""
    stamp = data_stamp(mode, demo_set)
    key = result_cache.result_key(mode, demo_set, stamp, params)
    mem = _memory()
    if key in mem:
        return mem[key]
    res = result_cache.load(key)
    if res is None:
        box = st.empty()
        with box.container():
            st.info("第一次用這組設定，AI 模型滾動訓練中。算完會存檔，之後打開就不用再等。"
                    "**計算中請不要切換頁面或調整左側設定**，否則會從頭重算。")
            bar = st.progress(0.0, text="準備資料…")

```

```
151         def progress(i, n, d):
152             bar.progress(min(i / max(n, 1), 1.0), text=f"訓練與預測 {i + 1}/{n}
    （{pd.Timestamp(d):%Y-%m}）")
153 
154         try:
155             res = run(_load(mode, demo_set, stamp), params, progress=progress)
156         except Exception as e:
157             box.empty()
158             st.error(f"回測無法執行：{e}")
159             return None
160         box.empty()
161         result_cache.save(key, res)
162     if len(mem) > 8:
163         mem.clear()
164     mem[key] = res
165     return res
```

程式 3　src/cache.py：回測存檔的鍵值 data_stamp()、result_key()（第 41 – 50、53 – 56 行）

```
41  def data_stamp(mode: str, demo_set: Optional[str] = None) -> Tuple:
42      """資料檔的名稱＋內容指紋；任何一個變了，存檔就失效。"""
43      if mode == "demo":
44          folder = config.DEMO_DIR / (demo_set or "")
45          files = sorted(folder.glob("*.csv"))
46      else:
47          files = [config.PRICES_FILE, config.BENCH_FILE, config.VALUATION_FILE,
    config.LISTING_FILE,
48                   config.INSTITUTIONAL_FILE,
49                   *sorted(config.TEJ_DIR.glob("*.*")),
    *sorted(config.LOCAL_DIR.glob("*.csv"))]
50      return tuple((f.name, _content_hash(f)) for f in files if f.exists() and f.name !=
    ".gitkeep")
    …（第 51 – 52 行略）
53  def result_key(mode: str, demo_set: Optional[str], stamp: Tuple, params) -> str:
54      flags = (config.USE_MARKET_FEATURES, config.TARGET_RELATIVE)      # 影響模型、但不在側邊欄的
    設定
55      raw = repr((CACHE_VERSION, mode, demo_set, stamp, sorted(asdict(params).items()),
    flags))
56      return hashlib.sha1(raw.encode("utf-8")).hexdigest()[:16]
```

程式 4　src/_fresh.py：雲端更新後重新載入模組 ensure_fresh()（第 14 – 24 行）

```
14  def ensure_fresh() -> None:
15      me = sys.modules[__name__]
16      stamp = _stamp()
17      if getattr(me, "_last", None) != stamp:
18          pkg = sys.modules.get("src")
19          for name in [m for m in list(sys.modules) if m.startswith("src.") and m !=
    __name__]:
20              del sys.modules[name]
21              short = name.split(".")[1]
22              if pkg is not None and short != "_fresh" and hasattr(pkg, short):
23                  delattr(pkg, short)     # 「from src import config」會先拿套件屬性，舊的也要拿掉
24          me._last = stamp
```

表 3　側邊欄與共用流程對照

畫面／功能	程式位置	說明
資料來源（真實資料／合成示範資料）	src/ui/common.py 第 77 – 90 行	選擇資料來源並載入 Dataset；真實資料模式會先檢查是否需要補抓股價
AI 模型、Top N、權重方式、ESG 比重、風險控制、進階設定	src/ui/common.py 第 93 – 133 行（程式 1）	回傳 Params；所有頁面共用同一組設定
短期轉弱減碼（2026 年 11 月新增，預設關閉）	src/ui/common.py 第 116 – 120 行；src/backtest.py 第 271 – 287 行	加權指數距 20 日高點跌逾 5% 時持股減半；可與風險控制同時使用，取較低的持股比例

畫面／功能	程式位置	說明
回測結果與進度條	src/ui/common.py 第 136–165 行（程式 2）	記憶體 → 存檔 → 計算；第一次計算時顯示訓練進度
回測存檔	src/cache.py 第 41–50、53–56 行（程式 3）	資料內容雜湊＋參數＋程式版本 → 存檔檔名
雲端更新後的模組重新載入	src/_fresh.py 第 14–24 行（程式 4）	src 內任一程式檔更新即清除已載入模組

表 3 對照側邊欄各項設定與各頁共用運算流程所在之程式位置；表中行數可直接對應程式檔，「程式 n」為本節所列之程式碼清單。

三、首頁（app.py）

首頁顯示資料狀態、最新一次選股名單、三種策略與比較基準的績效指標卡，以及樣本外期間的累積淨值圖。最新選股以資料最後一天的收盤價計算，並標示最近一次與下一次的調倉日。

🌿 ESG-AI 台股選股平台

AI 選股 × ESG 因子 × 台股大盤／ETF 比較｜左側選擇資料來源與策略參數，其他頁面會沿用同一組設定

› 📦 資料狀態與更新

TEJ ESG：1980 檔、8 期評等（最早公告日 2022-11-01）。

🗓️ 最新選股（2026-10-30 收盤後）

調倉固定**每 2 週**一次：最近一次 **2026-10-27**，下一次預定 **2026-11-10**（遇假日順延）。下面是用最新收盤價算出的名單，供下次調倉參考。

樣本外回測期間：2022-11-01 ～ 2026-10-30｜每 2 週調倉｜等權｜Top 20｜模型 Random Forest｜目標最高報酬（不做風險控制）

AI+ESG 年化報酬	單純 AI 年化報酬	單純 ESG 年化報酬	^TWII 加權指數 年化報酬
78.00%	111.76%	42.78%	38.32%
累積 813.95%｜最大回撤 -43.24%｜Sharpe 2.12	累積 1679.69%｜最大回撤 -45.59%｜Sharpe 2.72	累積 292.25%｜最大回撤 -23.63%｜Sharpe 2.13	累積 246.79%｜最大回撤 -28.69%

🧭 看大盤趨勢

AI+ESG

AI 預測排名與 ESG 排名加權

代號	公司	權重	AI 預測	TESG
2308.TW	台達電	5.0%	+1.99%	78.6
6488.TWO	環球晶	5.0%	+1.80%	72.8
6861.TW	睿生光電	5.0%	+1.23%	74.5
2303.TW	聯電	5.0%	+1.25%	73.2
3665.TW	貿聯-KY	5.0%	+2.02%	67.1
2892.TW	第一金	5.0%	+0.49%	75.3
2801.TW	彰銀	5.0%	+0.31%	75.0
3035.TW	智原	5.0%	+0.37%	72.5
3363.TWO	上詮	5.0%	+1.69%	60.3
6643.TWO	M31	5.0%	+0.64%	65.4

單純 AI

只看 AI 預測，不用 ESG

代號	公司	權重	AI 預測	TESG
7828.TWO	創新服務	5.0%	+2.14%	37.6
3665.TW	貿聯-KY	5.0%	+2.02%	67.1
2308.TW	台達電	5.0%	+1.99%	78.6
6488.TWO	環球晶	5.0%	+1.80%	72.8
3363.TWO	上詮	5.0%	+1.69%	60.3
7717.TWO	萊德光電-KY	5.0%	+1.58%	41.0
6834.TW	天二科技	5.0%	+1.55%	54.0
6693.TWO	廣閎科	5.0%	+1.32%	53.4
3016.TW	嘉晶	5.0%	+1.31%	50.7
7610.TW	聯友金屬-創	5.0%	+1.30%	56.6

單純 ESG

只看 ESG 總分

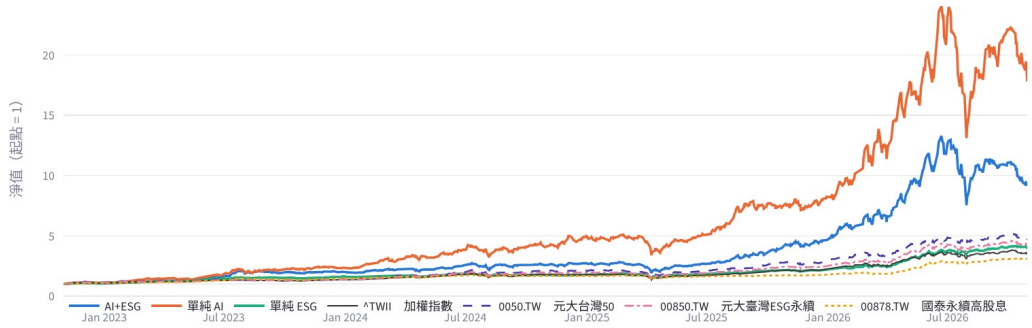
代號	公司	權重	AI 預測	TESG
2356.TW	英業達	5.0%	-0.46%	81.8
2412.TW	中華電	5.0%	+0.13%	80.4
2330.TW	台積電	5.0%	-0.46%	78.9
5347.TWO	世界	5.0%	-0.27%	78.7
2308.TW	台達電	5.0%	+1.99%	78.6
2886.TW	兆豐金	5.0%	+0.06%	76.5
2395.TW	研華	5.0%	-0.24%	76.5
2884.TW	玉山金	5.0%	+0.16%	76.3
2345.TW	智邦	5.0%	-0.22%	76.0
3264.TWO	欣銓	5.0%	-0.20%	75.8

AI 預測超額＝模型預估這檔股票未來 10 個交易日（約 2 週）比股票池平均多賺幾 %；**TESG**＝調倉日當時最新已公告的 TEJ ESG 總分（0～100）。三個策略用同一套欄位，差別只在挑選方式：單純 AI 看 AI 預測、單純 ESG 看 TESG、AI+ESG 兩者排名加權。

累積淨值（樣本外，扣除交易成本）

25

圖 3　首頁（1/2）



實線 = 策略；虛線 = 比較基準。詳細績效、持股比例、歷次持股請看「策略回測」頁；產業趨勢、法人買賣超請看「產業與籌碼」頁（僅供參考，不影響選股）。

免責聲明：本站為學術專題成果，所有數字均為歷史資料的回測結果，不代表未來績效，也不構成任何投資建議。回測假設以收盤價成交，未考慮滑價、流動性與稅務個別差異；股票池僅含目前仍上市櫃的公司，存在存活者偏差。資料來源：Yahoo Finance（股價）、台灣證券交易所（清單與估值）、TEJ 台灣經濟新報（TESG 永續評等）。

圖 3　首頁（2/2）

圖 3 呈現網站首頁，由上而下依序為資料狀態、最新一次選股名單與調倉日、三個策略與加權指數之年化報酬指標，以及樣本外期間之累積淨值圖，可據以掌握各策略之最新持股與累積績效。

程式 5　app.py：績效指標卡、最新選股與累積淨值（第 94－138、140－140 行）

```
params = sidebar_strategy(ds)
res = get_result(mode, demo_set, params)
if res is None:
    st.stop()
names = ds.names
labels = portfolio_labels(names)
show_notes([n for n in res.notes if n not in ds.notes])

st.subheader(f"🗓️ 最新選股（{res.latest_date:%Y-%m-%d} 收盤後）")
if res.rebalance_dates:
    _last_rb = pd.Timestamp(res.rebalance_dates[-1])
    _next_rb = bt.next_rebalance(res.rebalance_dates[0], _last_rb)
    st.markdown(f"調倉固定**每 {config.REBALANCE_WEEKS} 週**一次：最近一次 **{_last_rb:%Y-%m-%d}**，下一次預定 **{_next_rb:%Y-%m-%d}**（遇假日順延）。"
                + ("今天就是調倉日。" if _last_rb == pd.Timestamp(res.latest_date) else "下面是用最新收盤價算出的名單，供下次調倉參考。"))
st.caption(f"樣本外回測期間：{res.test_start:%Y-%m-%d} ～ {res.latest_date:%Y-%m-%d}｜每 {config.REBALANCE_WEEKS} 週調倉｜"
           f"{params.weighting}｜Top {params.top_n}｜模型 {params.model_name}｜"
           f"{'風險控制：避開跌破年線與最震盪的股票、大盤跌破年線持股減半' if params.risk_control else '目標最高報酬（不做風險控制）'}"
           f"{'｜短期轉弱減碼：大盤距 20 日高點跌逾 5% 持股減半' if params.pullback_control else ''}")

perf = res.perf.set_index("portfolio")
cards = []
for strat in STRATEGIES:
    if strat in perf.index:
        cards.append((f"{strat} 年化報酬", fmt_pct(perf.loc[strat, "ann_return"]),
                      f"累積 {fmt_pct(perf.loc[strat, 'cum_return'])}｜最大回撤 {fmt_pct(perf.loc[strat, 'max_drawdown'])}"
                      f"｜Sharpe {fmt_num(perf.loc[strat, 'sharpe'])}"))
bench_key = next((b for b in config.BENCHMARKS if b in perf.index), None)
```

```
if bench_key:
    cards.append((f"{labels.get(bench_key, bench_key)} 年化報酬", fmt_pct(perf.loc[bench_key, "ann_return"]),
                  f"累積 {fmt_pct(perf.loc[bench_key, 'cum_return'])}｜最大回撤 {fmt_pct(perf.loc[bench_key, 'max_drawdown'])}"))
if (params.risk_control or params.pullback_control) and not res.risk_log.empty:
    expo = res.risk_log[res.risk_log["date"] == res.risk_log["date"].max()]["exposure"].max()
    rule = "、".join(([] if not params.risk_control else ["跌破年線"]) + ([] if not params.pullback_control else ["距 20 日高點跌逾 5%"]))
    cards.append(("目前建議持股比例", fmt_pct(expo), "大盤未觸發減碼條件" if expo >= 0.99 else f"大盤觸發減碼（{rule}），持股減半"))
metric_cards(cards, min_width=200)

st.page_link("pages/0_大盤趨勢預測.py", label="看大盤趨勢", icon="🧭")
pick_info = stock_info(res, ds, res.latest_date)
pick_cols = st.columns(len(STRATEGIES))
for col, strat in zip(pick_cols, STRATEGIES):
    with col:
        st.markdown(f"**{strat}**<br><span style='opacity:.7;font-size:.9em'>{STRATEGY_DESC[strat]}</span>",
                    unsafe_allow_html=True)
        holdings_block(res.latest[res.latest["portfolio"] == strat], names, strat, info=pick_info)
st.caption(HOLDINGS_NOTE)
…（第 139–139 行略）
st.plotly_chart(charts.nav_chart(res.returns, labels, "累積淨值（樣本外，扣除交易成本）"), width="stretch")
```

程式 6　src/ui/charts.py：累積淨值圖 nav_chart()（第 88 – 118 行）

```
def nav_chart(returns: pd.DataFrame, labels: Optional[Dict[str, str]] = None, title: str = "累積淨值",
              log_y: bool = False) -> go.Figure:
    labels = labels or {}
    df = returns.sort_values("date")
    from .. import config
    rest = [p for p in df["portfolio"].unique() if p not in STRATEGIES]
    rest.sort(key=lambda p: config.BENCHMARKS.index(p) if p in config.BENCHMARKS else 99)  # 基準固定順序
    order = [s for s in STRATEGIES if s in set(df["portfolio"])] + rest
    smap = style_map(order)
    fig = go.Figure()
    lows, highs = [], []
    for p in order:
        g = df[df["portfolio"] == p]
        nav = (1 + g["ret"].fillna(0)).cumprod()
        if len(nav):
            lows.append(float(nav.min()))
            highs.append(float(nav.max()))
        c, dash, width = smap[p]
        fig.add_trace(go.Scatter(
            x=g["date"], y=nav, mode="lines", name=labels.get(p, p),
            line=dict(color=c, width=width, dash=dash),
            hovertemplate="%{y:.3f}",
        ))
    fig.add_hline(y=1, line=dict(color=GRID, width=1))
    fig = _layout(fig, title, y_title="淨值（起點 = 1，對數刻度）" if log_y else "淨值（起點 = 1）")
    if log_y:
        nice = [0.5, 0.7, 1, 1.5, 2, 3, 5, 7, 10, 15, 20, 30, 50, 70, 100, 150, 200, 300, 500, 700, 1000]
        lo, hi = (min(lows), max(highs)) if lows else (1.0, 1.0)
        ticks = [v for v in nice if lo * 0.9 <= v <= hi * 1.1] or [1]
        fig.update_yaxes(type="log", tickvals=ticks, ticktext=[f"{v:g}" for v in ticks])
    return fig
```

表 4　首頁畫面與程式對照

畫面區塊	程式位置	說明
資料狀態與更新（股票數、資料日期、ESG 期數、比較基準）	app.py 第 22 – 37 行	讀取 Dataset 並以 metric_cards() 顯示；可手動更新資料

畫面區塊	程式位置	說明
最新選股（調倉日、下次調倉日）	app.py 第 94–138 行（程式 5）	bt.next_rebalance() 推算下一次調倉日
各策略年化報酬指標卡	app.py 第 94–138 行（程式 5）	取自 Result.perf（metrics.performance_table）
三種策略的持股名單	src/ui/common.py 第 243–252 行	holdings_block() 顯示代號、名稱、權重與分數
累積淨值圖	app.py 第 140 行；src/ui/charts.py 第 88–118 行（程式 6）	實線為策略、虛線為比較基準；扣除交易成本

表 4 對照首頁各畫面區塊與其對應之程式檔、行數及功能。

四、大盤趨勢頁（pages/0_大盤趨勢預測.py）

本頁以 7 項常用指標描述加權指數目前的多空狀態，並以底色標示歷史上每一天依同一套規則的判斷結果。判斷只使用當天收盤為止的資料，美股與 VIX 使用前一晚已收盤的數值；本頁為描述性資訊，未用於選股與回測。頁面下方另提供未來一個月的波動與漲跌機率模型作為參考，其樣本外檢驗顯示漲跌方向的預測力接近隨機，因此判斷以指標表為主。

🧭 大盤趨勢

資料到 2026-10-30 收盤（美股、VIX 用前一晚已收盤的數字）

目前趨勢

🐂 牛市

自 2026-04-01 起，已 143 個交易日

指標統計

多 6 | 空 0

7 項指標，加總分數 +6

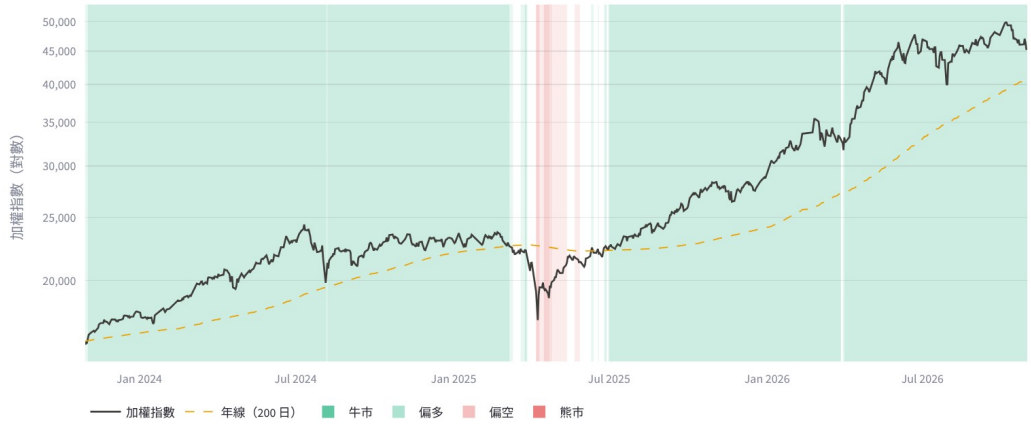
指標	目前數值	判斷	判斷標準
年線位置	加權指數 45,212，年線 40,692（+11.1%	🟢 多	站上年線（200 日均線）為多，跌破為空
年線方向	年線近 20 日 +5.10%	🟢 多	年線上揚為多、下彎為空，持平為中性
均線排列	季線 46,541 > 年線 40,692	🟢 多	季線（60 日）在年線之上為多頭排列
距一年高點	-9.3%	🟢 多	跌幅 10% 以內為多；10～20% 為修正（中性）；超過 20% 為熊市
近 60 日漲跌	+4.9%	⚪ 中性	漲 ≥ 5% 為多、跌 ≥ 10% 為空
美股趨勢	S&P 500 距年線 +5.8%	🟢 多	前一晚 S&P 500 在年線之上為多
VIX 恐慌指數	17.2	🟢 多	低於 20 市場平靜為多、高於 30 恐慌為空

判斷規則：7 項指標各給 多（+1）／中性（0）／空（−1）。加總 ≥ 4 為**牛市**、2～3 **偏多**、−1～1 **盤整**、≤ −2 **偏空**；距一年高點跌超過 20% 依慣例直接判為**熊市**。這張表描述「現在」的狀態，不是預測未來。

加權指數與歷來趨勢判斷

期間

○ 近 1 年 ◉ 近 3 年 ○ 近 5 年 ○ 2008 年起



底色＝當天收盤後依同一套規則的判斷（深綠牛市、淺綠偏多、淺紅偏空、深紅熊市）。2008 年以來各狀態所占比例：牛市 65%、偏多 8%、盤整 8%、偏空 9%、熊市 11%。

› 未來 1 個月展望（模型預測，僅供參考）

› 指標與資料說明

圖 4　大盤趨勢頁（1/2）

圖 4　大盤趨勢頁（2/2）

圖 4 呈現大盤趨勢頁。上方表格以 7 項指標判斷加權指數當前之多空狀態，各指標分別判定為多、中性或空；下方圖以底色標示歷史上各交易日依相同規則之判斷結果，可據以觀察市場狀態之轉換。本頁僅描述當前狀態，未用於選股。

程式 7　src/market_trend.py：7 項指標的每日多空判斷 trend_series()（第 222 – 260 行）

```
222 def trend_series(wide: pd.DataFrame) -> pd.DataFrame:
223     """每天收盤後的趨勢判斷（只用當天為止的資料）：7 項常見指標各給 多(+1)／中性(0)／空(−1)，加總後分級。
224     距一年高點跌超過 20% 依慣例直接判為熊市。"""
225     px = wide["^TWII"].dropna()
226     px = px[px > 0]
227     days = px.index
228     ma60, ma200 = px.rolling(60).mean(), px.rolling(200).mean()
229     t = pd.DataFrame(index=days)
230     t["close"], t["ma60"], t["ma200"] = px, ma60, ma200
231     t["gap200"] = px / ma200 - 1
232     t["slope200"] = ma200 / ma200.shift(20) - 1
233     t["dd250"] = px / px.rolling(250, min_periods=120).max() - 1
234     t["r60"] = px.pct_change(60)
235 
236     def lag(col):
237         if col not in wide:
238             return pd.Series(np.nan, index=days)
239         s = wide[col].dropna().rename("v").rename_axis("d").reset_index()
240         x = pd.merge_asof(pd.DataFrame({"d": days}), s, on="d", allow_exact_matches=False)
241         return pd.Series(x["v"].to_numpy(), index=days)
242 
243     spx = lag("^GSPC")
244     t["spx_gap200"] = spx / spx.rolling(200).mean() - 1
245     t["vix"] = lag("^VIX")
246     sig = pd.DataFrame(index=days)
247     sig["年線位置"] = np.sign(t["gap200"])
248     sig["年線方向"] = np.where(t["slope200"] > 0.002, 1, np.where(t["slope200"] < -0.002, -1,
    0))
249     sig["均線排列"] = np.sign(t["ma60"] - t["ma200"])
250     sig["距一年高點"] = np.where(t["dd250"] >= -0.10, 1, np.where(t["dd250"] <= -0.20, -1, 0))
251     sig["近 60 日漲跌"] = np.where(t["r60"] >= 0.05, 1, np.where(t["r60"] <= -0.10, -1, 0))
252     sig["美股趨勢"] = np.sign(t["spx_gap200"])
253     sig["VIX 恐慌指數"] = np.where(t["vix"] < 20, 1, np.where(t["vix"] > 30, -1, 0))
254     valid = t[["gap200", "slope200", "dd250", "r60"]].notna().all(axis=1)
255     score = sig.fillna(0).sum(axis=1)
256     verdict = pd.Series(np.select([t["dd250"] <= -0.20, score >= 4, score >= 2, score >= -
    1],
257                                   ["熊市", "牛市", "偏多", "盤整"], "偏空"), index=days)
258     t["score"] = score
259     t["verdict"] = verdict.where(valid)
260     return pd.concat([t, sig.add_prefix("sig_")], axis=1)
```

程式 8　pages/0_大盤趨勢預測.py：趨勢判斷表與歷史底色（第 58 – 67、80 – 85 行）

```
58 st.caption(f"資料到 {tr['date']:%Y-%m-%d} 收盤（美股、VIX 用前一晚已收盤的數字）")
59 metric_cards([
60     ("目前趨勢", f"{ICON[tr['verdict']]} {tr['verdict']}", f"自 {tr['since']:%Y-%m-%d} 起，已
    {tr['days']} 個交易日"),
61     ("指標統計", f"多 {tr['n_bull']}｜空 {tr['n_bear']}", f"7 項指標，加總分數 {tr['score']:
    +d}"),
62 ], min_width=260)
63 st.dataframe(tr["table"], hide_index=True, width="stretch",
64              column_config={"指標": st.column_config.TextColumn(width="small"),
65                             "判斷": st.column_config.TextColumn(width="small")})
66 st.caption("判斷規則：7 項指標各給 多（+1）／中性（0）／空（−1）。加總 ≥ 4 為**牛市**、2～3 **偏多**、
    −1～1 **盤整**、"
67            "≤ −2 **偏空**；距一年高點跌超過 20% 依慣例直接判為**熊市**。這張表描述「現在」的狀態，不是預
    測未來。")
…（第 68 – 79 行略）
80 seg = (ts["verdict"] != ts["verdict"].shift()).cumsum()
81 for _, g in ts.groupby(seg):
82     v = g["verdict"].iloc[0]
83     if COLOR.get(v, "rgba(0,0,0,0)").endswith("0.0)"):
84         continue
85     fig.add_vrect(x0=g.index[0], x1=g.index[-1] + pd.Timedelta(days=1), fillcolor=COLOR[v],
    line_width=0, layer="below")
```

表 5　大盤趨勢頁畫面與程式對照

畫面區塊	程式位置	說明
目前趨勢、指標統計	pages/0_大盤趨勢預測.py 第 58–67 行（程式 8）	current_trend() 的結論、持續天數與多空指標數
7 項指標判斷表	src/market_trend.py 第 263–288 行	年線位置、年線方向、均線排列、距一年高點、近 60 日漲跌、美股趨勢、VIX
判斷規則（加總分數分級）	src/market_trend.py 第 222–260 行（程式 7）	各指標 +1／0／−1 加總：≥ 4 牛市、2～3 偏多、−1～1 盤整、≤ −2 偏空；距一年高點跌逾 20% 為熊市
加權指數與歷來趨勢判斷圖	pages/0_大盤趨勢預測.py 第 80–85 行（程式 8）	對數刻度；底色為當天的判斷
未來 1 個月展望（參考）	src/market_trend.py 第 170–209 行	羅吉斯迴歸滾動訓練，2008 年起樣本外 AUC 與 Brier 分數

表 5 對照大盤趨勢頁之判斷表、歷史趨勢圖與展望模型所在之程式位置，並說明 7 項指標之判斷規則。

五、策略回測頁（pages/1_策略回測.py）

本頁有四個分頁：「績效比較」可選擇期間比較累積淨值、績效指標、風險報酬散佈與回撤；「歷次持股」列出每個調倉日三種策略的持股，並以調倉日當天的模型預估對照持有期間的實際走勢（2026 年 11 月起，預估的大盤部分改依調倉日的大盤狀態估計，並顯示期末虧損機率）；「換股與成本」列出每次調倉的週轉率與交易成本；「方法說明」以文字說明預測目標、特徵、訓練方式與回測規則。

策略回測

樣本外期間 2022-11-01 ～ 2026-10-30，共 105 次調倉（每 2 週一次）

TEJ ESG：1980 檔、8 期評等（最早公告日 2022-11-01）。

績效比較　歷次持股　換股與成本　方法說明

期間

全部樣本外期間　近 1 年　近 3 年　自訂　　對數刻度

全部樣本外期間 累積淨值



投組	累積報酬	年化報酬	年化波動	Sharpe	最大回撤	Beta	詹森 Alpha	日報酬平均	日報酬變異數	交易日數
AI+ESG	813.95%	78.00%	36.26%	2.124	-43.24%	1.209	31.84%	0.26%	0.000522	967
單純 AI	1679.69%	111.76%	40.66%	2.724	-45.59%	1.223	65.20%	0.33%	0.000656	967
單純 ESG	292.25%	42.78%	19.63%	2.128	-23.63%	0.789	12.27%	0.15%	0.000153	967
0050.TW　元大台灣50	368.50%	49.49%	24.14%	2.009	-27.48%	1.053	9.33%	0.17%	0.000231	968
00850.TW　元大臺灣ESG永續	315.90%	44.92%	22.08%	1.989	-26.91%	0.974	7.68%	0.16%	0.000194	968
00878.TW　國泰永續高股息	204.95%	33.68%	16.86%	1.938	-22.29%	0.642	8.80%	0.12%	0.000113	968
^TWII　加權指數	246.79%	38.32%	22.11%	1.688	-28.69%	1.000	-0.00%	0.14%	0.000194	966

下載績效表 CSV

風險與報酬　　**回撤（相對歷史高點）**

圖 5　策略回測頁：績效比較（1/2）



圖 5　策略回測頁：績效比較（2/2）

圖 5 呈現策略回測頁之績效比較分頁，包括各策略與比較基準之累積淨值、績效指標、風險報酬散佈與回撤，並可選擇不同評估期間，據以比較各策略之報酬、波動與最大跌幅。

程式 9　pages/1_策略回測.py：績效比較分頁（期間選擇、淨值圖、績效表）（第 40 – 74 行）

```
with tab1:
    r0, r1 = res.test_start, res.latest_date
    options = {"全部樣本外期間": (r0, r1)}
    for label, years in [("近 1 年", 1), ("近 3 年", 3), ("近 5 年", 5)]:
        a0 = r1 - pd.DateOffset(years=years)
        if a0 > r0:
            options[label] = (a0, r1)
    options["自訂"] = None
    p1, p2 = st.columns([5, 1])
    keys = list(options)
    choice = p1.radio("期間", keys, horizontal=True)
    log_y = p2.checkbox("對數刻度", value=False, help="期間很長、淨值差距很大時，對數刻度比較看得出每段期間的漲跌幅")
    if choice == "自訂":
        rng = st.date_input("選擇區間", value=(r0.date(), r1.date()), min_value=r0.date(), max_value=r1.date())
        if not isinstance(rng, (list, tuple)) or len(rng) < 2:
            st.info("請選擇結束日期")
            st.stop()
        a, b = pd.Timestamp(rng[0]), pd.Timestamp(rng[1])
    else:
        a, b = options[choice]
    full = choice == "全部樣本外期間"
    sub_ret = res.returns if full else res.returns[(res.returns["date"] > a) & (res.returns["date"] <= b)]
    if sub_ret.empty:
        st.info("這段期間沒有資料")
        st.stop()
    sub_perf = res.perf if full else performance_table(sub_ret)
    title = "全部樣本外期間　累積淨值" if full else f"累積淨值（{a:%Y-%m-%d} ～ {b:%Y-%m-%d}）"
    st.plotly_chart(charts.nav_chart(sub_ret, labels, title, log_y=log_y), width="stretch")
    tbl = format_table(sub_perf)
    tbl["投組"] = tbl["投組"].map(lambda p: labels.get(p, p))
    st.dataframe(tbl, hide_index=True, width="stretch")
    st.download_button("下載績效表 CSV", to_csv_bytes(sub_perf), "performance.csv", "text/csv")
```

```
    c1, c2 = st.columns(2)
    c1.plotly_chart(charts.risk_return_scatter(sub_perf, labels), width="stretch")
    c2.plotly_chart(charts.drawdown_chart(sub_ret, labels), width="stretch")
```

程式 10　src/metrics.py：績效指標 performance() 與 CAPM capm()（第 36 – 58、66 – 75 行）

```
def performance(returns: pd.Series, rf: float = config.RISK_FREE_RATE) -> Dict[str, float]:
    r = pd.Series(returns, dtype=float).dropna()
    if r.empty:
        return {k: np.nan for k in METRIC_LABELS}
    wealth = (1 + r).cumprod()
    cum = wealth.iloc[-1] - 1
    ann_ret = (1 + cum) ** (config.TRADING_DAYS / len(r)) - 1 if cum > -1 else -1.0
    ann_vol = r.std(ddof=1) * np.sqrt(config.TRADING_DAYS) if len(r) > 1 else np.nan
    sharpe = (ann_ret - rf) / ann_vol if ann_vol and ann_vol > 0 else np.nan
    peak = np.maximum(wealth.cummax(), 1.0)
    mdd = float(min((wealth / peak - 1).min(), 0.0))
    return {
        "cum_return": float(cum),
        "ann_return": float(ann_ret),
        "ann_vol": float(ann_vol) if pd.notna(ann_vol) else np.nan,
        "sharpe": float(sharpe) if pd.notna(sharpe) else np.nan,
        "max_drawdown": mdd,
        "beta": np.nan,
        "jensen_alpha": np.nan,
        "daily_mean": float(r.mean()),
        "daily_var": float(r.var(ddof=1)) if len(r) > 1 else np.nan,
        "n_days": int(len(r)),
    }
…（第 59 – 65 行略）
def capm(port: pd.Series, market: pd.Series, rf: float = config.RISK_FREE_RATE) -> Dict[str, float]:
    """port、market：以日期為 index 的日報酬。回傳 beta 與年化詹森 Alpha。"""
    df = pd.concat([port.rename("p"), market.rename("m")], axis=1).dropna()
    if len(df) < 20 or df["m"].var() == 0:
        return {"beta": np.nan, "jensen_alpha": np.nan}
    rf_d = (1 + rf) ** (1 / config.TRADING_DAYS) - 1
    ex_p, ex_m = df["p"] - rf_d, df["m"] - rf_d
    beta = float(np.cov(ex_p, ex_m, ddof=1)[0, 1] / ex_m.var(ddof=1))
    alpha = _ann(df["p"]) - (rf + beta * (_ann(df["m"]) - rf))
    return {"beta": beta, "jensen_alpha": float(alpha)}
```

策略回測

樣本外期間 2022-11-01 ～ 2026-10-30，共 105 次調倉（每 2 週一次）

TEJ ESG：1980 檔、8 期評等（最早公告日 2022-11-01）。

績效比較　歷次持股　換股與成本　方法說明

調倉日（每 2 週一次，遇假日順延）

2026-10-27（週二）→ 11/10　持有中

持有期間：2026-10-27 收盤買進 → 2026-11-10 調倉（共 10 個交易日，尚未走完）

- 調倉日（10/27）大盤狀態 **牛市回檔**：過去同狀態之後 10 個交易日，加權指數平均 +1.2%、上漲機率 61%（306 天）
- 最新一天大盤狀態 **牛市回檔**：過去同狀態之後 10 個交易日，加權指數平均 +1.2%、上漲機率 61%（306 天）

大盤狀態以「大盤趨勢」頁的 7 項指標判斷；牛市或偏多但距 20 日高點跌逾 5% 標為「回檔」。大盤短期漲跌方向的預測力很低（樣本外 AUC 約 0.5），上漲機率只反映歷史上同狀態之後的實際比例，不代表模型能預測方向。

AI+ESG

代號	公司	權重	AI 預測	TESG
2892.TW	第一金	5.0%	+0.46%	75.3
3665.TW	貿聯-KY	5.0%	+1.26%	67.1
6488.TWO	環球晶	5.0%	+0.37%	72.8
5876.TW	上海商銀	5.0%	+0.52%	71.1
2884.TW	玉山金	5.0%	+0.16%	76.3
2801.TW	彰銀	5.0%	+0.20%	75.0
4938.TW	和碩	5.0%	+0.24%	73.1
3045.TW	台灣大	5.0%	+0.12%	74.8
2492.TW	華新科	5.0%	+0.92%	64.1
2886.TW	兆豐金	5.0%	+0.08%	76.5

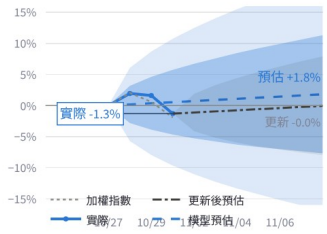
單純 AI

代號	公司	權重	AI 預測	TESG
3055.TW	蔚華科	5.0%	+1.46%	43.5
3665.TW	貿聯-KY	5.0%	+1.26%	67.1
3441.TWO	聯一光	5.0%	+1.20%	43.9
3406.TW	玉晶光	5.0%	+1.14%	53.4
6199.TWO	天品	5.0%	+1.09%	49.4
6672.TW	騰輝電子-KY	5.0%	+1.02%	53.3
6456.TW	GIS-KY	5.0%	+1.00%	52.2
2340.TW	台亞	5.0%	+0.92%	58.2
2492.TW	華新科	5.0%	+0.92%	64.1
1709.TW	和益	5.0%	+0.92%	58.6

單純 ESG

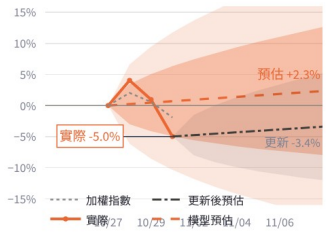
代號	公司	權重	AI 預測	TESG
2356.TW	英業達	5.0%	-0.30%	81.8
2412.TW	中華電	5.0%	+0.07%	80.4
2330.TW	台積電	5.0%	-0.27%	78.9
5347.TWO	世界	5.0%	-0.19%	78.7
2308.TW	台達電	5.0%	-0.28%	78.6
2886.TW	兆豐金	5.0%	+0.08%	76.5
2395.TW	研華	5.0%	-0.22%	76.5
2884.TW	玉山金	5.0%	+0.16%	76.3
2345.TW	智邦	5.0%	-0.41%	76.0
3264.TWO	欣銓	5.0%	-0.05%	75.8

持有期間走勢（累積報酬）



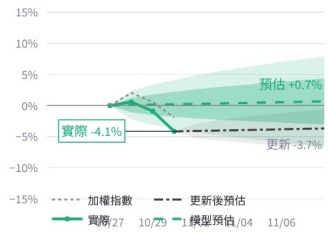
預估期末　期末虧損機率

持有期間走勢（累積報酬）



預估期末　期末虧損機率

持有期間走勢（累積報酬）



預估期末　期末虧損機率

圖 6　策略回測頁：歷次持股與持有期間預估（1/2）

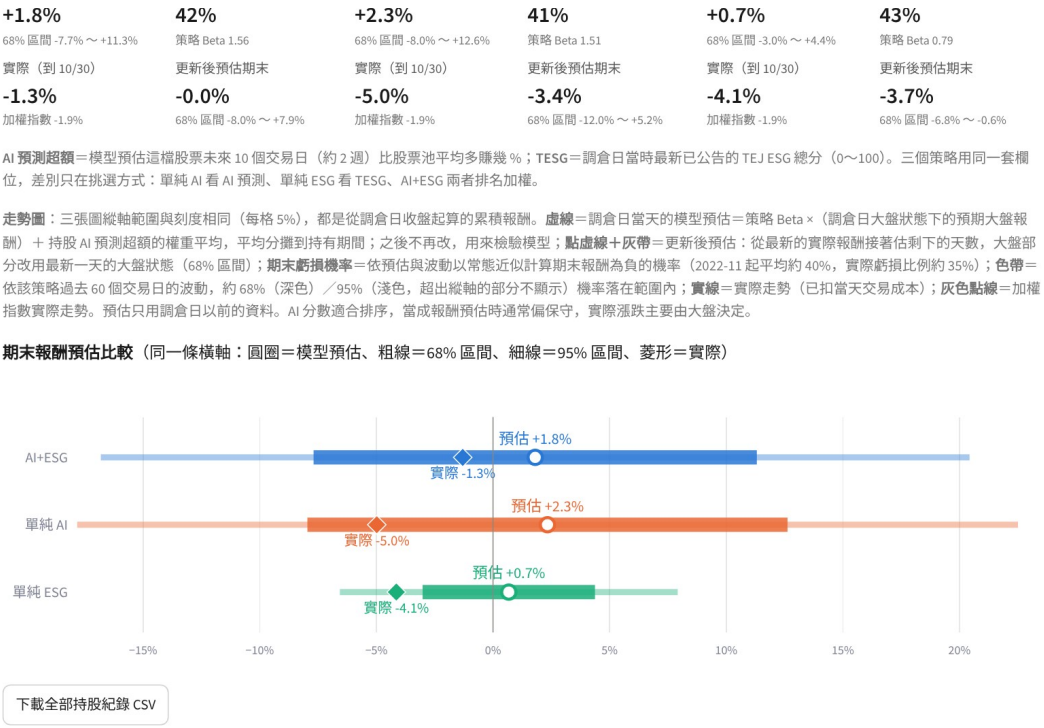


圖 6　策略回測頁：歷次持股與持有期間預估（2/2）

圖 6 呈現策略回測頁之歷次持股分頁。上方列示調倉日與最新交易日之大盤狀態及歷史上漲機率；中段為三個策略於該調倉日之持股；下方走勢圖比較調倉日之預估（虛線與色帶）與實際走勢（實線），指標卡列示預估期末報酬、期末虧損機率與實際報酬，可據以檢驗模型預估與實際結果之差異。

程式 11　pages/1_策略回測.py：歷次持股：三種策略的持有期間預估與大盤狀態（第 111 – 135 行）

```
111         # 先算三個策略的預估，再用「同一個縱軸範圍」畫圖，單位與刻度一致才能直接比較
112         wide = forecast.market_wide(ds.benchmarks, _macro(config.MACRO_FILE.stat().st_mtime)
    if config.MACRO_FILE.exists() else None)
113         fcs, subs, fc_info = {}, {}, {}
114         for strat in STRATEGIES:
115             sub = res.holdings[(res.holdings["date"] == d) & (res.holdings["portfolio"] ==
    strat)]
116             subs[strat] = sub
117             if not sub.empty and len(window):
118                 fc = forecast.holding_forecast(sub, res.evaluation, res.returns,
    ds.benchmarks, strat, d,
119                                                window, params.hold_days, wide=wide)
120                 fcs[strat], fc_info[strat] = fc["path"], fc
121         st.markdown(f"**持有期間**：{d:%Y-%m-%d} 收盤買進 → {end:%Y-%m-%d} 調倉（共
    {len(window)} 個交易日"
122                     + ("，尚未走完" if d == dates[0] and end > pd.Timestamp(res.latest_date)
    else "") + "）")
123         if fc_info:
124             any_fc = next(iter(fc_info.values()))
125             v, vu = any_fc["view"], any_fc["view_upd"]
126
127             def view_text(x, when):
128                 if not x or x["state"] is None:
```

```
                    return f"{when}：大盤狀態資料不足，大盤部分改用長期平均年化 {config.MARKET_DRIFT_ANNUAL:.0%}"
                return (f"{when}大盤狀態 **{x['state']}**：過去同狀態之後 {params.hold_days} 個交易日，加權指數平均 "
                        f"{x['cond_mean']:+.1%}、上漲機率 {x['up_prob']:.0%}（{x['n']:,} 天）")
            st.markdown("- " + view_text(v, f"調倉日（{d:%m/%d}）") +
                        ("\n- " + view_text(vu, "最新一天") if vu else ""))
            st.caption("大盤狀態以「大盤趨勢」頁的 7 項指標判斷；牛市或偏多但距 20 日高點跌逾 5% 標為「回檔」。"
                       "大盤短期漲跌方向的預測力很低（樣本外 AUC 約 0.5），上漲機率只反映歷史上同狀態之後的實際比例，不代表模型能預測方向。")
```

程式 12　src/forecast.py：持有期間預估路徑、區間與期末虧損機率 holding_forecast()（第 184 – 240 行）

```
def holding_forecast(holdings: pd.DataFrame, evaluation: pd.DataFrame, returns: pd.DataFrame,
                     bench: pd.DataFrame, strategy: str, d: pd.Timestamp, window: pd.DatetimeIndex,
                     hold_days: int = config.HOLD_DAYS, wide: Optional[pd.DataFrame] = None) -> Dict:
    """回傳 {'path': DataFrame(date, step, expected, lo68, hi68, lo95, hi95, actual, market, upd...),
            'excess': 預期超額, 'drift': 大盤預期日報酬, 'beta': 策略 Beta, 'sigma': 日波動,
            'view': 調倉日的大盤狀態, 'view_upd': 最新實際日的大盤狀態（更新後預估用）}；actual／market 只有已發生的日子才有值。
    wide：大盤狀態用的收盤價寬表（market_wide()）；沒給時用 bench 的加權指數。"""
    d = pd.Timestamp(d)
    wide = wide if wide is not None else market_wide(bench)
    excess = expected_excess(holdings, evaluation, d)
    view = market_view(wide, d, hold_days)
    drift = view["drift"]
    beta = strategy_beta(returns, strategy, d)
    mkt_r = returns[returns["portfolio"] == config.MARKET_INDEX].set_index("date")["ret"]
    mkt_sigma = float(mkt_r[mkt_r.index <= d].tail(60).std()) if len(mkt_r[mkt_r.index <= d]) >= 20 else None
    sigma = strategy_vol(returns, strategy, d, fallback=mkt_sigma * 1.3 if mkt_sigma else None)

    steps = np.arange(0, len(window) + 1)
    dates = [d] + list(window)
    base = beta * ((1 + drift) ** steps - 1)
    exp = base + (excess or 0.0) * steps / max(hold_days, 1)
    spread = sigma * np.sqrt(steps)
    path = pd.DataFrame({"date": dates, "step": steps, "expected": exp,
                         "lo68": exp - Z68 * spread, "hi68": exp + Z68 * spread,
                         "lo95": exp - Z95 * spread, "hi95": exp + Z95 * spread})

    def realized(name: str) -> pd.Series:
        r = returns[(returns["portfolio"] == name) & (returns["date"] > d)].set_index("date")["ret"]
        r = r.reindex(window).dropna()
        cum = (1 + r).cumprod() - 1
        return pd.concat([pd.Series([0.0], index=[d]), cum])

    act, mkt = realized(strategy), realized(config.MARKET_INDEX)
    path["actual"] = path["date"].map(act) if len(act) > 1 else np.nan
    path["market"] = path["date"].map(mkt) if len(mkt) > 1 else np.nan

    # 更新後預估：從最新的實際值往後估剩下的天數
    for c in ["upd", "upd_lo68", "upd_hi68"]:
        path[c] = np.nan
    known = path["actual"].dropna()
    view_upd = None
    if 1 < len(known) < len(path):
        k = int(known.index[-1])
        a_k = float(known.iloc[-1])
        view_upd = market_view(wide, path.loc[k, "date"], hold_days)     # 用最新一天收盤後的大盤狀態
        rest = path.index[path.index >= k]
        n = (path.loc[rest, "step"] - path.loc[k, "step"]).to_numpy()
        upd = (1 + a_k) * (1 + beta * ((1 + view_upd["drift"]) ** n - 1)) - 1 + (excess or 0.0) * n / max(hold_days, 1)
        path.loc[rest, "upd"] = upd
        path.loc[rest, "upd_lo68"] = upd - Z68 * sigma * np.sqrt(n)
        path.loc[rest, "upd_hi68"] = upd + Z68 * sigma * np.sqrt(n)
```

```
    # 期末虧損機率：以常態近似 P(期末報酬 < 0)；2022-11 起回測的平均值約 39%～42%，實際虧損比例約 35%
    from math import erf, sqrt
    sd_end = sigma * sqrt(max(len(window), 1))
    p_loss = 0.5 * (1 + erf((0 - float(exp[-1])) / (sd_end * sqrt(2)))) if sd_end > 0 else
float(exp[-1] < 0)
    return {"path": path, "excess": excess, "drift": drift, "beta": beta, "sigma": sigma,
"view": view,
            "view_upd": view_upd, "p_loss": p_loss}
```

程式 13　src/forecast.py：依大盤狀態的預期大盤報酬 market_view() 與策略 Beta strategy_beta()（第 98 – 123、126 – 134 行）

```
def market_view(wide: Optional[pd.DataFrame], d: pd.Timestamp, hold_days: int =
config.HOLD_DAYS) -> Dict:
    """d 收盤後的大盤狀態與預期大盤日報酬（只用 d 以前已實現的 hold_days 日報酬）。
    回傳 state、n（過去同狀態天數）、up_prob（過去同狀態之後上漲的比例）、cond_mean／
all_mean（hold_days 日平均報酬）、
    drift（收縮後換算的日報酬）。資料不足時 state 為 None、drift 用長期平均。"""
    out = {"state": None, "n": 0, "up_prob": np.nan, "cond_mean": np.nan, "all_mean":
np.nan, "drift": market_drift()}
    if wide is None or config.MARKET_INDEX not in wide:
        return out
    ts = _trend(wide)
    d = pd.Timestamp(d)
    ts = ts[ts.index <= d]
    if ts.empty or pd.isna(ts["state"].iloc[-1]):
        return out
    now = ts["state"].iloc[-1]
    px = ts["close"]
    fwd = px.shift(-hold_days) / px - 1                      # 只有 d 以前已走完 hold_days 天的
才有值
    ok = fwd.notna() & ts["state"].notna()
    if ok.sum() < 250:
        return {**out, "state": now}
    same = fwd[ok & (ts["state"] == now)]
    all_mean = float(fwd[ok].mean())
    n = int(len(same))
    cond = float(same.mean()) if n else all_mean
    m = (n * cond + STATE_SHRINK * all_mean) / (n + STATE_SHRINK)
    return {"state": now, "n": n, "up_prob": float((same > 0).mean()) if n else np.nan,
            "cond_mean": cond, "all_mean": all_mean, "drift": (1 + m) ** (1 / hold_days) -
1,
            "dd20": float(ts["dd20"].iloc[-1]) if pd.notna(ts["dd20"].iloc[-1]) else np.nan}
…（第 124 – 125 行略）
def strategy_beta(returns: pd.DataFrame, strategy: str, d: pd.Timestamp, lookback: int =
120,
                  index: str = config.MARKET_INDEX) -> float:
    """策略到 d 為止最近 lookback 個交易日相對加權指數的 Beta（限制在 0.3～2.0）；資料不足時為 1。"""
    w = returns[returns["portfolio"].isin([strategy, index]) & (returns["date"] <= d)] \
        .pivot_table(index="date", columns="portfolio",
values="ret").dropna().tail(lookback)
    if strategy == index or len(w) < 40 or strategy not in w or index not in w or
w[index].var() == 0:
        return 1.0
    b = float(np.cov(w[strategy], w[index], ddof=1)[0, 1] / w[index].var(ddof=1))
    return float(np.clip(b, 0.3, 2.0))
```



策略回測

樣本外期間 2022-11-01 ～ 2026-10-30，共 105 次調倉（每 2 週一次）

TEJ ESG：1980 檔、8 期評等（最早公告日 2022-11-01）。

績效比較　歷次持股　換股與成本　方法說明

portfolio	換股次數	平均週轉率	累計成本
AI+ESG	105	51.5%	31.46%
單純 AI	105	61.7%	37.73%
單純 ESG	105	3.9%	2.27%

週轉率 = 換掉的部位比例（0% 完全沒換、100% 全部換掉）。成本已反映在淨值裡。

date	portfolio	turnover	cost
2026-10-27	AI+ESG	75.9%	0.444%
2026-10-27	單純 AI	52.1%	0.305%
2026-10-27	單純 ESG	3.6%	0.021%
2026-10-13	AI+ESG	75.3%	0.441%
2026-10-13	單純 AI	62.2%	0.364%
2026-10-13	單純 ESG	1.6%	0.009%
2026-09-29	AI+ESG	65.5%	0.383%
2026-09-29	單純 AI	67.1%	0.393%
2026-09-29	單純 ESG	3.0%	0.018%
2026-09-15	AI+ESG	61.5%	0.360%

圖 7　策略回測頁：換股與成本

圖 7 呈現策略回測頁之換股與成本分頁。上表彙整各策略之換股次數、平均週轉率與累計交易成本，下表列示各次調倉之週轉率與成本；由圖可知，AI 策略之換股頻率與交易成本明顯高於規則式 ESG 篩選。

程式 14　pages/1_策略回測.py：換股與成本分頁（第 192 – 203 行）

```
192 with tab4:
193     if res.trades.empty:
194         st.info("沒有換股紀錄")
195     else:
196         summary = res.trades.groupby("portfolio").agg(換股次數=("turnover", "size"), 平均週轉率
    =("turnover", "mean"),
197                                                       累計成本=("cost", "sum")).reindex(
198             [s for s in STRATEGIES if s in set(res.trades["portfolio"])])
199         st.dataframe(summary.style.format({"平均週轉率": "{:.1%}", "累計成本": "{:.2%}"}),
    width="stretch")
200         st.caption("週轉率 = 換掉的部位比例（0% 完全沒換、100% 全部換掉）。成本已反映在淨值裡。")
201         st.dataframe(res.trades.sort_values(["date", "portfolio"], ascending=[False, True])
202                      .style.format({"turnover": "{:.1%}", "cost": "{:.3%}", "date": "{:%Y-
    %m-%d}"}),
203                      hide_index=True, width="stretch")
```

程式 15　src/backtest.py：交易成本 trading_cost()（第 180 – 186 行）

```
180 def trading_cost(old: pd.Series, new: pd.Series) -> float:
181     """買進付手續費；賣出付手續費 + 證交稅。old 是換股前（已漂移）的權重。"""
182     idx = old.index.union(new.index)
183     diff = new.reindex(idx, fill_value=0) - old.reindex(idx, fill_value=0)
184     buys, sells = diff.clip(lower=0).sum(), (-diff).clip(lower=0).sum()
185     slip = getattr(config, "SLIPPAGE", 0.0)           # 每邊滑價／額外成本（例如 0.001 = 10
    bps）
186     return float(buys * (config.COMMISSION + slip) + sells * (config.COMMISSION +
    config.SELL_TAX + slip))
```

📈 策略回測

樣本外期間 2022-11-01 ～ 2026-10-30，共 105 次調倉（每 2 週一次）

TEJ ESG：1980 檔、8 期評等（最早公告日 2022-11-01）。

績效比較　歷次持股　換股與成本　方法說明

流程

資料（股價、估值、ESG）→特徵工程→AI 模型滾動訓練→三種策略選股→回測→與大盤／ETF 比較（含詹森 Alpha）

預測目標（Y）

每檔股票從調倉日收盤起，未來 **10 個交易日**的報酬，再減去同一天所有股票的平均（相對報酬）。 模型學的是「哪些股票會比別人好」，不是猜大盤漲跌。

特徵（X）

20 日年化波動率、月線乖離率、近 1 月報酬（動能）、季線乖離率、近 3 月報酬（動能）、近 5 日報酬（動能）、短期反轉（5 日 × 60 日報酬）、量能比（5 日均量 ÷ 60 日均量）、股價淨值比、距 20 日最高價跌幅、本益比、殖利率、距 20 日最高價天數、連續下跌天數（共 14 個）。**AI 模型不使用 ESG**，ESG 在選股階段才與 AI 分數融合。

其中「距 20 日最高價跌幅、距 20 日最高價天數、連續下跌天數、量能比、短期反轉」是**見頂訊號**， 用來區分「還在噴的飆股」和「已經從高點回落的飆股」。加入後「單純 AI」年化從約 100% 提高到約 106%（3 組隨機種子平均）。

訓練方式：滾動式（walk-forward）

- 每 3 次調倉（約 6 週）重新訓練一次，只用「在訓練當天已經知道結果」的樣本 （樣本日期 ≤ 調倉日往前 10 個交易日）；訓練樣本每 5 個交易日取一天。
- 缺值用訓練集的中位數補，不會用到測試期資訊。
- **訓練資料從 2017-08 起；樣本外績效從 2022-11-01 起**（TEJ ESG 評等第一次公告日）， 三個策略從同一天開始比較才公平。在那之前的資料只拿來訓練模型，不計入績效。

三種策略

1. **單純 AI**：模型預測分數最高的 20 檔。
2. **AI+ESG**：先排除爭議分數 > 3、EPS 為負的公司，混合分數 = 70% × AI 排名 ＋ 30% × ESG 排名。
3. **單純 ESG**：同樣的排除條件後，ESG 總分最高的 20 檔。

目標：最高報酬（不限制回撤）

預設設定以**報酬最大化**為唯一目標，不考慮最大回撤：

1. **股票池 300 檔**：近 20 個交易日平均成交金額最大的上市櫃普通股（市值前 50 大一定納入）。
2. **選前 20 檔、等權**：永遠滿倉，不設停損、不因大盤轉弱減碼。
3. **每 2 週調倉**（固定同一個星期幾，遇假日順延到下一個交易日），模型每 3 次調倉重新訓練一次。
4. **怎麼選出這組設定**：在近兩年半（300 檔）比較 Top N、等權／集中加權、調倉與重訓頻率； 再用 2018 年底起約 8 年的回測比較特徵與產業的用法（期間較長、包含 2020 與 2022 年的大跌），各用 3 組隨機種子重跑， 選平均最好、且不是只有單一參數特別好的組合。

產業

圖 8　策略回測頁：方法說明（1/2）

選股**不限制同一產業的檔數**，也不排除任何產業：只要個股分數夠高就選。產業資訊放在「產業與籌碼」頁供參考。 我們測試過四種「把產業納入選股」的做法（2018 年底起約 8 年回測），**報酬都比不考慮產業低**，所以預設都不採用：

- 產業趨勢加分（AI 排名與產業近 1 月報酬排名加權）：產業權重 0% 約 110%、5% 約 104%、10% 約 98%、20% 約 85%，越加越差；改用產業近 3／6 個月報酬或產業季線乖離也一樣。
- 把產業趨勢當成模型特徵：約 85%（不加時約 100%）。
- 只買產業在季線之上的股票：約 78%。
- 同一產業最多 5 檔：報酬略降（約 106%→105%），回撤較小，但會限制選股，已取消。

可能原因：強勢股常在產業轉強之前就先漲，等產業趨勢確認時已經太晚；而過去一個月最強的產業，接下來也常出現漲多拉回。

代價：持股集中在高波動股票，最大回撤約 −40%，比大盤深；調倉次數多，交易成本也較高（已反映在報酬裡）。

風險控制（選用） 目前關閉

側邊欄勾選後會加上三條規則，可降低回撤，但報酬也會明顯變低：

1. **不買跌破年線的股票**：收盤價在 200 日均線之下就不買。
2. **不買最震盪的 20%**：近 60 日波動最高的 20% 不買。
3. **大盤跌破年線持股減半**：加權指數在 200 日均線之下時，持股降為 50%、其餘放現金。

短期轉弱減碼（選用） 目前關閉

「跌破年線」反應較慢，牛市中的急跌回檔常常不會觸發。勾選後，加權指數收盤價距近 20 個交易日最高收盤價 跌幅超過 5% 時，持股降為 50%，回到 5% 以內恢復滿倉（每天收盤檢查、只調整比例不換股）。 可減少回檔時的虧損與最大回撤，代價是報酬降低、換倉次數增加；可與上面的風險控制同時使用，取較低的持股比例。

回測規則

- **每 2 週**調倉一次（從回測起點起算，固定同一個星期幾，遇假日順延），以當天收盤價換股，報酬從**下一個交易日**開始計算。
- 持有期間權重隨股價漂移，等權配置；未投入的部分為現金（報酬 0）。
- 交易成本：買進手續費 0.1425%、賣出手續費 0.1425% + 證交稅 0.3%，換股與調整持股比例都會扣。
- ESG 與估值資料一律依**公告日**對齊：某天只會用到當天以前已公告的資料。

限制

- 股票池是用「最近」的成交金額挑出的目前上市櫃公司，存在存活者偏差與前視偏差（過去冷門、現在才熱門的股票也被納入）；以收盤價成交為假設，未考慮滑價。
- 樣本外期間從 2022-11-01 起，主要是 AI／電子股大多頭；預設參數是在 2018 年起的期間比較後選出的，未來報酬很可能比回測低。
- 2017～2022 年的本益比／淨值比若還沒補齊，該期間以中位數代替（每日排程會自動補）。

圖 8　策略回測頁：方法說明（2/2）

圖 8 呈現策略回測頁之方法說明分頁，以文字說明預測目標、特徵、訓練方式、三種策略、回測規則與研究限制，其參數依側邊欄當前設定自動帶入。

表 6　策略回測頁畫面與程式對照

畫面區塊	程式位置	說明
期間選擇、累積淨值（可切換對數刻度）	pages/1_策略回測.py 第 40–74 行（程式 9）	全部樣本外期間或近 1／3／5 年、自訂區間
績效表（年化報酬、波動、Sharpe、最大回撤、Beta、詹森 Alpha）	src/metrics.py 第 36–58、66–75、88–101 行（程式 10）	幾何年化；CAPM 以加權指數為市場
風險報酬散佈、回撤圖	src/ui/charts.py 第 121–132、135–150 行	Plotly 圖，顏色固定對應策略
歷次持股（調倉日選單、三策略持股）	pages/1_策略回測.py 第 92–109 行	調倉日每 2 週一次，遇假日順延

畫面區塊	程式位置	說明
持有期間走勢圖與預估指標卡	pages/1_策略回測.py 第 111–189 行（程式 11）；src/forecast.py 第 184–240 行（程式 12）	預估 = 策略 Beta × 依大盤狀態的預期大盤報酬＋持股 AI 預測超額；區間依過去 60 日波動；另顯示期末虧損機率
大盤狀態與歷史上漲機率（預估上方的說明）	src/forecast.py 第 98–123、126–134 行（程式 13）	7 項趨勢指標判斷狀態（牛市回檔另外標示）；取過去同狀態之後 10 個交易日的平均報酬並向全期平均收縮；只用調倉日以前已實現的資料
期末報酬預估比較	src/ui/charts.py 第 326–352 行	圓圈＝預估、粗線＝68% 區間、菱形＝實際
換股次數、週轉率、累計成本	pages/1_策略回測.py 第 192–203 行（程式 14）；src/backtest.py 第 180–186 行（程式 15）	買進手續費 0.1425%；賣出手續費＋證交稅 0.3%
方法說明（文字）	pages/1_策略回測.py 第 205–281 行	依目前設定動態帶入參數

表 6 對照策略回測頁四個分頁之畫面區塊與程式位置；其中持有期間預估與大盤狀態二列為 2026 年 11 月更新後之做法。

六、個股分析頁（pages/2_個股分析.py）

本頁可輸入任一股票代號；不在資料中的股票會即時向 Yahoo Finance 下載。頁面上方為累積報酬、年化波動、Sharpe、最大回撤與 ESG 評等，下方分為走勢、風險與技術指標、ESG、同業比較四個分頁。ESG 評等取「期間結束時已公告」的最新一期，不使用之後才公告的資料。



圖 9　個股分析頁：走勢（以 2330 台積電為例）

圖 9 呈現個股分析頁之走勢分頁。上方指標列示個股於選定期間之累積報酬、年化波動、夏普值、最大回撤與 ESG 評等，下方為股價與均線走勢，以及依固定門檻自動產生之走勢摘要。



圖 10　個股分析頁：風險與技術指標

圖 10 呈現個股分析頁之風險與技術指標分頁。左圖為 20 日年化波動率之變化，右圖為日報酬分布，下表說明各風險指標之意義，可據以判斷個股之波動程度與極端報酬之頻率。



圖 11　個股分析頁：ESG

圖 11 呈現個股分析頁之 ESG 分頁。左側雷達圖呈現 E、S、G 三構面分數，右表列示 TESG 總分、等級、爭議相關分數與公告日，下方為歷次評等之變化；所呈現者僅限期間結束時已公告之評等。



圖 12　個股分析頁：同業比較

圖 12 呈現個股分析頁之同業比較分頁。上表比較同產業股票之 ESG 總分、報酬、波動、夏普值與最大回撤，下方散佈圖呈現 ESG 總分與累積報酬之關係，可據以觀察該股於同業中之相對位置。

程式 16　pages/2_個股分析.py：技術指標與已公告的 ESG 評等（第 58 – 71 行）

```
d = px_df.copy()
d["sma20"] = d["close"].rolling(20).mean()
d["sma60"] = d["close"].rolling(60).mean()
d["ret"] = d["close"].pct_change()
d["vol20"] = d["ret"].rolling(20).std() * np.sqrt(252)
d = d[d["date"] >= start]
m = price_metrics(d["close"])

# ESG：取期間結束時「已公告」的最新一期
esg_row = None
if not ds.esg.empty:
    e = ds.esg[(ds.esg["ticker"] == ticker)].sort_values("available_date")
    e_known = e[e["available_date"].isna() | (e["available_date"] <= d["date"].max())]
    esg_row = (e_known if not e_known.empty else e).iloc[-1] if not e.empty else None
```

程式 17　pages/2_個股分析.py：同業比較（第 148 – 157 行）

```
    for t in dict.fromkeys([ticker, *peers]):
        g = ds.prices[(ds.prices["ticker"] == t) & (ds.prices["date"] >= start)].sort_values("date")
        if t == ticker:
            g = d
        if len(g) < 20:
            continue
        pm = performance(g["close"].pct_change())
        rows.append({"代號": t, "公司": names.get(t, ""), "產業": industries.get(t, ""),
                     "ESG 總分": latest_esg.get(t, np.nan), "累積報酬": pm["cum_return"], "年化波動": pm["ann_vol"],
                     "Sharpe": pm["sharpe"], "最大回撤": pm["max_drawdown"]})
```

表 7　個股分析頁畫面與程式對照

畫面區塊	程式位置	說明
代號輸入與即時下載	pages/2_個股分析.py 第 19 – 23、27 – 32 行	資料中沒有的股票以 yahoo.fetch_prices() 下載，快取 1 小時
指標卡（累積報酬、波動、Sharpe、最大回撤、ESG）	pages/2_個股分析.py 第 58 – 71 行（程式 16）	metrics.price_metrics()；ESG 取期末已公告的一期
走勢（價格、均線、規則式摘要）	pages/2_個股分析.py 第 92 – 102 行	摘要依固定門檻自動產生
風險與技術指標	pages/2_個股分析.py 第 104 – 114 行	20 日年化波動、日報酬分布與指標說明
ESG（雷達圖、評等明細、歷史）	pages/2_個股分析.py 第 116 – 138 行	TEJ TESG 總分與 E／S／G、爭議分數、公告日
同業比較	pages/2_個股分析.py 第 148 – 157 行（程式 17）	同產業至少 3 檔時比較同產業，否則比較全部股票

表 7 對照個股分析頁之代號輸入、指標與四個分頁所在之程式位置。

七、模型解釋頁（pages/3_模型解釋.py）

本頁說明 AI 模型的表現與使用的特徵：上方為平均 Rank IC、IC 為正的比例與 Top N 每期平均超額報酬；左圖為最後一次訓練的特徵重要性，右圖為每次調倉的 Rank IC；下方可展開 SHAP 分析、特徵清單與每次訓練的紀錄。



圖 13　模型解釋頁

圖 13 呈現模型解釋頁。上方指標列示平均 Rank IC、IC 為正之比例與前 N 檔平均超額報酬；左圖為最近一次訓練之特徵重要性，右圖為各調倉日之 Rank IC，可據以觀察模型所依賴之特徵及其排序能力之穩定性。

程式 18　pages/3_模型解釋.py：Rank IC 與 Top N 超額報酬（第 28 – 40 行）

```
ic = rank_ic(res.evaluation)
ev = res.evaluation.dropna(subset=["pred", "fwd_ret"])
cards = [("平均 Rank IC", fmt_num(ic["rank_ic"].mean(), 3) if not ic.empty else "-",
          "預測排名與實際報酬排名的相關；0.03～0.05 就算有用"),
         ("IC > 0 的比例", fmt_pct((ic["rank_ic"] > 0).mean()) if not ic.empty else "-")]
if not ev.empty:
    hits = []
    for _, g in ev.groupby("date"):
        top = g.nlargest(params.top_n, "pred")
        hits.append(top["fwd_ret"].mean() - g["fwd_ret"].mean())
    cards.append((f"Top {params.top_n} 平均超額報酬（每期）", fmt_pct(np.mean(hits)), "前 N 檔比全部股票多賺多少（未扣成本）"))
cards.append(("可評估的調倉次數", f"{ic['date'].nunique() if not ic.empty else 0}"))
metric_cards(cards, min_width=190)
```

程式 19　src/pipeline.py：每次調倉的 Rank IC rank_ic()（第 202 – 209 行）

```
def rank_ic(evaluation: pd.DataFrame) -> pd.DataFrame:
    """每個調倉日：預測分數與實際報酬的 Spearman 相關（Rank IC）。"""
    ev = evaluation.dropna(subset=["pred", "fwd_ret"])
    rows = []
    for d, g in ev.groupby("date"):
        if len(g) >= 3:
            rows.append({"date": d, "rank_ic": g["pred"].rank().corr(g["fwd_ret"].rank())})
    return pd.DataFrame(rows, columns=["date", "rank_ic"])
```

表 8　模型解釋頁畫面與程式對照

畫面區塊	程式位置	說明
平均 Rank IC、IC > 0 比例、Top N 平均超額	pages/3_模型解釋.py 第 28 – 40 行（程式 18）；src/pipeline.py 第 202 – 209 行（程式 19）	Spearman 等級相關；超額 = 前 N 檔平均 − 全部平均（未扣成本）
特徵重要性圖	pages/3_模型解釋.py 第 43 – 49 行	Random Forest 內建重要性
Rank IC 時間序列	src/ui/charts.py 第 254 – 261 行	每次調倉一點
SHAP 分析	pages/3_模型解釋.py 第 54 – 67 行	需另行安裝 shap；按鈕觸發計算
使用的特徵	pages/3_模型解釋.py 第 69 – 79 行	技術面、見頂訊號、估值；ESG 不進入模型，於選股時融合
訓練紀錄	pages/3_模型解釋.py 第 81 – 121 行	每次訓練的資料期間、樣本數與負責期間的表現

表 8 對照模型解釋頁之 Rank IC、特徵重要性、SHAP 分析與訓練紀錄所在之程式位置。

八、產業與籌碼頁（pages/4_產業與籌碼.py）

本頁提供產業趨勢與三大法人買賣超資訊，僅供參考，不影響選股與回測。產業指數為股票池內同產業股票的等權平均；法人資料取自證交所 T86 報表（僅涵蓋上市股票）。



圖 14　產業與籌碼頁：產業趨勢（1/2）



產業	檔數	近5日	近1月	季線乖離	趨勢向上
食品工業	1	0.8%	8.1%	5.2%	☑
水泥工業	1	1.4%	7.8%	12.2%	☑
金融保險業	15	1.1%	7.6%	10.1%	☑
貿易百貨業	1	-0.2%	6.1%	3.3%	☑
資訊服務業	2	0.9%	5.4%	8.9%	☑
汽車工業	1	0.8%	4.9%	2.0%	☑
居家生活	1	-1.6%	4.0%	-1.1%	☐
玻璃陶瓷	1	-7.2%	4.0%	3.9%	☑
塑膠工業	4	-2.3%	4.0%	8.8%	☑
紡織纖維	3	-2.7%	3.9%	0.5%	☑

圖 14　產業與籌碼頁：產業趨勢（2/2）

圖 14 呈現產業與籌碼頁之產業趨勢分頁。上方指標列示趨勢向上之產業數及近 1 個月表現最強與最弱之產業，長條圖與下表列示各產業之近期報酬與季線乖離；本頁僅供參考，未用於選股。

產業與籌碼

資料日期：2026-10-30 收盤｜本頁僅供參考，不影響選股與回測

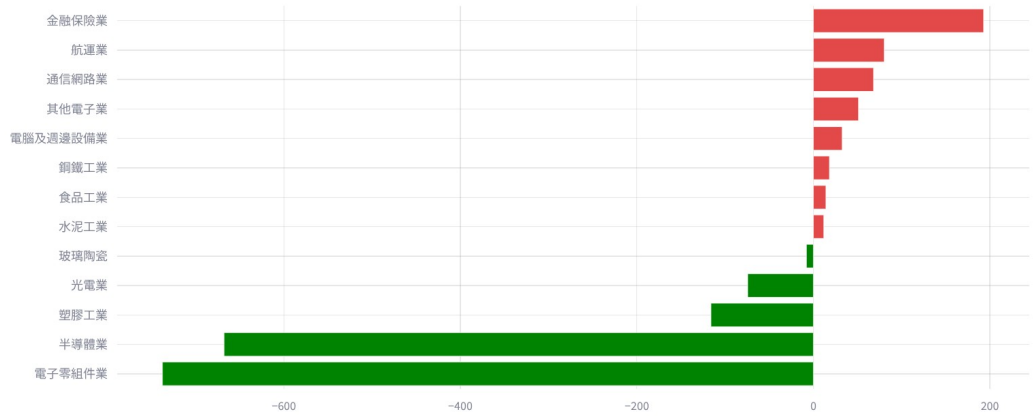
產業趨勢　主力動向（三大法人）

統計期間

○ 近 1 個交易日　● 近 5 個交易日　○ 近 20 個交易日

2026-10-22 ～ 2026-10-30｜單位：張（1 張 = 1,000 股）｜金額 = 買超股數 × 最新收盤價（估計）

法人買賣超金額（依產業，億元）



買超金額前 15 名

代號	公司	產業	外資(張)	投信(張)	自營商(張)	合計
2344.TW	華邦電	半導體業	57,262	-819	3,123	59
2408.TW	南亞科	半導體業	17,760	-472	1,174	18
2317.TW	鴻海	其他電子業	17,960	2,675	2,330	22
2603.TW	長榮	航運業	14,138	1,566	1,806	17
6770.TW	力積電	半導體業	48,466	7,390	-704	55
2891.TW	中信金	金融保險業	36,196	3,736	8,079	48
2412.TW	中華電	通信網路業	18,983	2,317	279	21
3443.TW	創意	半導體業	-23	326	93	
2059.TW	川湖	電子零組件業	160	158	-18	
7769.TW	鴻勁	半導體業	-77	550	16	

賣超金額前 15 名

代號	公司	產業	外資(張)	投信(張)	自營商(張)	合計
2330.TW	台積電	半導體業	-26,131	6,912	4,006	-15
2308.TW	台達電	電子零組件業	-19,926	-2,641	575	-21
2303.TW	聯電	半導體業	-115,793	-92,954	-8,202	-216
2454.TW	聯發科	半導體業	-9,096	2,623	732	-5
2327.TW	國巨*	電子零組件業	-23,323	1,077	-122	-22
1303.TW	南亞	塑膠工業	-43,075	5,464	1,792	-35
4958.TW	臻鼎-KY	電子零組件業	-23,024	1,995	-553	-21
3189.TW	景碩	半導體業	-10,149	2,444	-16	-7
6213.TW	聯茂	電子零組件業	-12,116	1,472	315	-10
2409.TW	友達	光電業	-147,785	6,952	-15,783	-156

投信買超前 10 名（投信常被視為中小型股的主力）

圖 15　產業與籌碼頁：主力動向（三大法人）（1/2）

代號	公司	產業	外資(張)	投信(張)	自營商(張)	合計(張)	估計金額(億)
3711.TW	日月光投控	半導體業	-26,270	19,542	1,233	-5,496	-36.7
0050.TW	元大台灣50		-222,557	15,035	-163,487	-371,009	None
2887.TW	台新新光金	金融保險業	18,117	11,082	9,204	38,403	16.2
2352.TW	佳世達	電腦及週邊設備業	-4,154	9,807	512	6,165	None
2492.TW	華新科	電子零組件業	-18,585	8,426	-1,084	-11,243	-42.8
2890.TW	永豐金	金融保險業	6,033	7,774	6,596	20,402	9.5
6770.TW	力積電	半導體業	48,466	7,390	-704	55,152	36.7
2883.TW	凱基金	金融保險業	-10,274	7,191	7,972	4,889	1.8
2409.TW	友達	光電業	-147,785	6,952	-15,783	-156,616	-48.0
2330.TW	台積電	半導體業	-26,131	6,912	4,006	-15,213	-360.6

資料來源：證交所 T86（僅上市股票）。外資 = 外陸資＋外資自營商。

圖 15　產業與籌碼頁：主力動向（三大法人）（2/2）

圖 15 呈現產業與籌碼頁之主力動向分頁。上圖依產業彙整三大法人之買賣超金額，下方表格列示買超與賣超金額前 15 名及投信買超前 10 名，可據以觀察近期法人資金之流向。

程式 20　src/analysis.py：產業趨勢 industry_trend()（第 12 – 30 行）

```
12  def industry_trend(prices: pd.DataFrame, industries: Dict[str, str], lookback_days: int =
    260) -> pd.DataFrame:
13      """每個產業最新一天的：檔數、近 5 日／近 1 月報酬、季線乖離、趨勢是否向上。"""
14      if prices.empty or not industries:
15          return pd.DataFrame()
16      days = np.sort(prices["date"].unique())[-lookback_days:]
17      tech = add_technical(prices[prices["date"].isin(days)], hold_days=1)
18      panel = add_industry(tech, industries)
19      last = panel["date"].max()
20      p5 = panel.dropna(subset=["industry"]).copy()
21      # 產業近 5 日報酬：用等權日報酬連乘
22      recent = p5[p5["date"] > np.sort(p5["date"].unique())[-6]]
23      r5 = recent.groupby(["industry", "date"])["ret_1d"].mean().groupby(level=0).apply(lambda
    s: (1 + s).prod() - 1)
24      snap = panel[(panel["date"] == last) & panel["industry"].notna()]
25      out = snap.groupby("industry").agg(檔數=("ticker", "nunique"), 近1月=("ind_mom_20",
    "first"),
26                                         季線乖離=("ind_gap_ma60", "first"), 趨勢向上=("ind_up",
    "first"))
27      out["近5日"] = r5.reindex(out.index)
28      out["趨勢向上"] = out["趨勢向上"] == 1
29      out = out.reset_index().rename(columns={"industry": "產業"})
30      return out[["產業", "檔數", "近5日", "近1月", "季線乖離", "趨勢向上"]].sort_values("近1月",
    ascending=False)
```

程式 21　src/analysis.py：三大法人買賣超彙整 flow_leaders()（第 33 – 56 行）

```
33  def flow_leaders(flows: pd.DataFrame, prices: pd.DataFrame, names: Dict[str, str],
34                   industries: Dict[str, str], days: int = 5) -> pd.DataFrame:
35      """最近 days 個交易日的三大法人買賣超（張、估計金額億元）。金額 = 股數 × 最新收盤價，僅供排序參考。"""
36      if flows is None or flows.empty:
37          return pd.DataFrame()
38      f = flows.copy()
39      f["date"] = pd.to_datetime(f["date"])
40      recent_days = np.sort(f["date"].unique())[-days:]
41      f = f[f["date"].isin(recent_days)]
42      agg = f.groupby("ticker")[["foreign_net", "trust_net", "dealer_net", "total_net"]].sum()
```

```
    close = (prices.sort_values("date").groupby("ticker")["close"].last() if not
prices.empty
             else pd.Series(dtype=float))
    out = pd.DataFrame({
        "代號": agg.index,
        "公司": [names.get(t, "") for t in agg.index],
        "產業": [industries.get(t, "") for t in agg.index],
        "外資(張)": (agg["foreign_net"] / 1000).round(0).values,
        "投信(張)": (agg["trust_net"] / 1000).round(0).values,
        "自營商(張)": (agg["dealer_net"] / 1000).round(0).values,
        "合計(張)": (agg["total_net"] / 1000).round(0).values,
    })
    out["估計金額(億)"] = (agg["total_net"].values * close.reindex(agg.index).values / 1e8)
    out.attrs["period"] = (pd.Timestamp(recent_days[0]), pd.Timestamp(recent_days[-1]))
    return out
```

表 9　產業與籌碼頁畫面與程式對照

畫面區塊	程式位置	說明
趨勢向上的產業數、近 1 月最強／最弱	pages/4_產業與籌碼.py 第 29–38 行；src/analysis.py 第 12–30 行（程式 20）	趨勢向上 = 產業指數在季線之上且近 20 日上漲
各產業近 1 個月報酬圖與明細表	pages/4_產業與籌碼.py 第 40–45 行	等權產業指數
法人買賣超金額（依產業）	src/analysis.py 第 59–64 行	估計金額 = 買超股數 × 最新收盤價
買超、賣超前 15 名與投信買超前 10 名	pages/4_產業與籌碼.py 第 49–78 行；src/analysis.py 第 33–56 行（程式 21）	可選近 1／5／20 個交易日

表 9 對照產業趨勢與三大法人買賣超各畫面區塊所在之程式位置及計算方式。

九、論文正式版本頁（pages/5_論文正式版本.py）

本頁呈現論文第三章與第四章的正式表格與圖。表格內容由 build_report.py 產生論文時一併寫入 web_tables.json，頁面讀取後以與論文相同的格式（深藍表頭、隔列淺底）顯示；論文中的表 3-3、3-4、3-8 與表 4-1 至 4-14（表 4-10 除外）即為本頁的截圖（由 capture_web.py 擷取）。因此網站與論文的數字出自同一份輸出檔，不會不一致。

論文正式版本

研究版本 thesis_2026-10-20｜資料截點 2026-10-20｜評估期 2022-11-01 ～ 2026-10-20（探索性）｜每 2 週調倉

本頁是論文使用的正式結果：每個調倉日以「當時」成交金額前 300 大重建股票池，並保留之後下市的股票；設定固定在 research_config.json。其他頁面以「目前」前 300 大回測，報酬會因後見之明偏誤而偏高，兩者數字不同。

單純 AI 年化報酬	AI+ESG 年化報酬	規則式 ESG 篩選 年化報酬	0050 年化報酬	00850 年化報酬
50.9%	45.7%	35.5%	50.8%	46.1%
Sharpe 1.24｜最大回撤 -46.9%	Sharpe 1.40｜最大回撤 -40.9%	Sharpe 1.74｜最大回撤 -24.6%	Sharpe 2.06｜最大回撤 -27.5%	Sharpe 2.05｜最大回撤 -26.9%

樣本與期間　投組績效　風險比較　統計檢定　穩健性測試

表 3-3 逐期股票池概況（各調倉日）

項目	平均	最小	最大
掛牌普通股檔數	1,900	1,821	1,978
入選股票池檔數	300	300	300
入選門檻（第 300 名 20 日均成交金額，億元）	2.64	0.98	7.21
入選股票中之後下市的檔數	3.2	0	11

註：普通股以代號規則判斷（4 碼、不以 0 開頭、排除 91xx 存託憑證），因此也適用已下市股票。研究期間曾進入股票池的股票共 1357 檔。逐期名單另存於 universe_history.csv。

表 3-4 樣本期間切分

期間	日期	用途	單純 AI 年化	加權指數年化
開發期	2017-08-01 ～ 2020-12-31	模型訓練；訓練樣本累積	50.1%	25.2%
驗證期	2021-01-01 ～ 2022-10-31	訓練延伸；觀察單純 AI 的表現	18.6%	−7.0%
探索性評估期	2022-11-01 ～ 2026-10-20	三策略與 ETF 比較（本文主要結果）	41.8%	39.7%

註：開發期與驗證期的報酬來自同一套滾動訓練、自 2019-01-02 起的長期間回測（ESG 資料自 2022-11 才有，因此只列單純 AI）；開發期前段為模型暖機，未計入。此長期間回測的調倉日期網格與正式版本不同，因此探索性評估期的數字與表 4-1 不同（差異見表 4-11 對照 2）。

表 3-8 參數試驗紀錄（探索性）

圖 16　論文正式版本頁（樣本與期間分頁）（1/2）

日期	項目	比較內容	評估期間	結果	選定
2026-10-02	AI 模型	Random Forest／XGBoost／兩者平均	2022-11～2026-10（3 組種子）	Random Forest 最高，其餘明顯較低	Random Forest
2026-10-02	持股數與權重	Top 10～30；等權／集中加權	2022-11～2026-10	等權 Top 10～30 約 116%～126%；集中加權約 90%	等權 Top 20
2026-10-02	風險控制	個股年線＋波動濾網＋大盤濾網 開／關	2022-11～2026-10	開啟後報酬明顯下降	關閉
2026-10-02	調倉頻率	每月 vs. 約每 2 週（每 10 個交易日）	2022-11～2026-10	約 108% → 約 119%	約每 2 週
2026-10-02	重新訓練頻率	每 3 次 vs. 每 6 次調倉	2022-11～2026-10	每 3 次高約 10 個百分點	每 3 次
2026-10-27	調倉日期規則	每 10 個交易日 vs. 每 2 週固定同一星期幾（遇休市順延）	—	週期相近；固定星期幾便於實務執行與追蹤	每 2 週固定同一星期幾
2026-10-03	見頂訊號特徵	加入與否	2018～2026（8 年）	約 100% → 約 106%	加入
2026-10-19	產業納入方式	產業特徵／只買趨勢向上產業／同產業上限 5 檔	2018～2026（8 年）	約 85%／約 78%／約 105%（不納入約 106%）	不納入
2026-10-19	產業趨勢加分	產業近 1 月報酬排名權重 0%／5%／10%／20%	2018～2026（8 年，1 組種子）	約 110%／104%／98%／85%	0%
—	ESG 權重	AI+ESG 中 ESG 排名比重	—	未系統性調整（沿用 30%）	30%

註：試驗結果為當時平台版本（以 Yahoo 現存 300 檔為股票池）的數字，與本研究正式版本不同，僅用來說明參數如何被選出。

› 下載與重現方式

圖 16　論文正式版本頁（樣本與期間分頁）（2/2）

圖 16 呈現論文正式版本頁，內容為論文第三、四章之正式表格與圖，上方指標列示各投組與 ETF 之年化報酬、夏普值與最大回撤；論文中之表格即為本頁之截圖，故網站與論文之數據一致。

程式 22　pages/5_論文正式版本.py：表格產生與分頁配置（第 58－64、67－75、140－162 行）

```
 58  def table_html(t) -> str:
 59      tot = sum(t["widths"])
 60      cols = "".join(f'<col style="width:{w / tot:.2%}">' for w in t["widths"])
 61      head = "".join(f"<th>{html.escape(h)}</th>" for h in t["header"])
 62      body = "".join("<tr>" + "".join(f'<td class="{t["align"][j]}">{html.escape(v)}</td>' for
     j, v in enumerate(r))
 63                     + "</tr>" for r in t["rows"])
 64      return f'{CSS}<table
     class="thesis-tbl"><colgroup>{cols}</colgroup><thead><tr>{head}</tr></thead><tbody>{body}</
     tbody></table>'
    …（第 65－66 行略）
 67  def show_table(key: str) -> None:
 68      t = tables.get(key)
 69      if t is None:
 70          return
 71      st.markdown(f"**{t['caption'].replace('  ', ' ')}**")
 72      with st.container(key=f"tbl-{key}"):
 73          st.html(table_html(t))
 74      if t.get("note"):
 75          st.caption(t["note"])
    …（第 76－139 行略）
140  NAMES = ["樣本與期間", "投組績效", "風險比較", "統計檢定", "穩健性測試"]      # 對應論文第三章第一節與第
     四章四節
141  # ?all=1：全部依序展開（給 capture_web.py 截圖用）；平常以分頁顯示
142  tabs = [st.container() for _ in NAMES] if st.query_params.get("all") else st.tabs(NAMES)
143  with tabs[0]:
144      for k in ["3-3", "3-4", "3-8"]:
145          show_table(k)
146  with tabs[1]:
147      show_table("4-1")
```

```
    show_figure("4-1", "圖 4-1 累積淨值（對數刻度）", nav_figure(),
                "起點 = 1（評估期第一天）；策略已扣手續費與證交稅。來源：strategy_returns.csv。")
    show_table("4-2")
    show_table("4-3")
    show_figure("4-2", "圖 4-2 各調倉日持股加權 TESG 與 E／S／G 分數", esg_figure(),
                "「股票池等權」為當期 300 檔等權，作為市場代理。來源：
esg_quality_by_rebalance.csv。")
with tabs[2]:
    for k in ["4-4", "4-5", "4-6"]:
        show_table(k)
with tabs[3]:
    for k in ["4-7", "4-8", "4-9", "4-10"]:
        show_table(k)
with tabs[4]:
    for k in ["4-11", "4-12", "4-13", "4-14"]:
        show_table(k)
```

表 10　論文正式版本頁畫面與程式對照

畫面區塊	程式位置	說明
版本與期間說明、績效指標卡	pages/5_論文正式版本.py 第 36–138 行	讀取 research_config.json 與 table_4_1_performance.csv
五個分頁的表格	pages/5_論文正式版本.py 第 58–64、67–75、140–162 行（程式 22）	樣本與期間、投組績效、風險比較、統計檢定、穩健性測試
圖 4-1 累積淨值、圖 4-2 ESG 分數	pages/5_論文正式版本.py 第 96–104、107–128 行	讀取 strategy_returns.csv 與 esg_quality_by_rebalance.csv
下載與重現方式	pages/5_論文正式版本.py 第 164–169 行	提供逐期股票池、持股、報酬與 manifest 下載
論文表格截圖	scripts/research/capture_web.py 第 45–81 行	以 ?all=1 依序展開全部表格後逐一截圖

表 10 對照論文正式版本頁各畫面區塊所在之程式位置，以及論文表格截圖之產生方式。

十、選股與回測核心程式（網站與論文共用）

pipeline.run() 串接整個流程：建立特徵面板 → 依逐期股票池篩選（論文版本）→ 產生調倉日 → 滾動訓練模型並預測 → 計算三種策略的分數與權重 → 模擬每日淨值並扣除交易成本 → 彙整績效。網站各頁與論文研究程式都呼叫這個函式，差別只在輸入的資料（Dataset）與設定。表 11 依執行順序列出各步驟所在的程式位置。

程式 23　src/pipeline.py：整體流程 run()（節錄）（第 82 – 95、97 – 116、126 – 138、148 – 167 行）

```
    panel, info = build_panel(prices, ds.esg, ds.financials, p.hold_days,
static_esg=p.static_esg,
                              industries=industries)
    if info["esg_static"]:
        notes.append("⚠️ ESG 以「最新一期」回填到所有歷史日期，回測含前視偏差，只能當對照組。")

    if config.USE_MARKET_FEATURES:
        panel = add_market_features(panel, ds.benchmarks)
    full_panel = panel
    elig = getattr(ds, "eligible", None)
    if elig is not None and not elig.empty:
        # 逐期股票池：每天只保留「當天以前 20 日成交金額前 N 大」的股票（訓練樣本與選股都適用）；
        # 已持有的股票之後掉出股票池仍會照實際報酬計算（用 full_panel 的報酬）。
        key = elig[["date", "ticker"]].assign(_in=1)
        panel = panel.merge(key, on=["date", "ticker"], how="inner").drop(columns="_in")
…（第 96 – 96 行略）
    trading_days = pd.DatetimeIndex(sorted(full_panel["date"].unique()))
    if len(trading_days) < 120:
        raise ValueError(f"只有 {len(trading_days)} 個交易日，資料太短，建議至少 1 年。")
    test_start = _first_test_day(trading_days, p.hold_days)
    bt_floor = pd.Timestamp(config.BACKTEST_START)
    if test_start < bt_floor < trading_days[-1] - pd.Timedelta(days=60):
        test_start = pd.Timestamp(trading_days[trading_days >= bt_floor][0])   # 績效從 ESG
有資料那天起算
    schedule = bt.rebalance_schedule(trading_days, test_start)
    last_day = pd.Timestamp(trading_days[-1])
    pred_dates = schedule + ([last_day] if last_day not in schedule else [])

    extra = [f for f in info.get("extra_features", [])
             if (config.USE_PEAK_FEATURES or f not in PEAK_FEATURES)
             and (config.USE_INDUSTRY_FEATURES or f not in INDUSTRY_FEATURES)]
    features = info["tech_features"] + extra + info["fin_features"]      # 模型不使用 ESG（ESG
另外融合）
    if config.USE_MARKET_FEATURES:
        features += [f for f in MARKET_FEATURES if f in panel.columns]
    wf = walk_forward(panel, features, pred_dates, trading_days, p.hold_days, p.model_name,
                      p.n_estimators, retrain_every=config.RETRAIN_EVERY,
required=info["tech_features"],
                      progress=progress)
…（第 117 – 125 行略）
    targets: Dict[str, Dict[pd.Timestamp, pd.Series]] = {s: {} for s in bt.STRATEGIES}
    hold_rows = []
    for d, snap in snaps.groupby("date"):
        if config.INDUSTRY_TREND_FILTER and "ind_gap_ma60" in snap.columns:
            snap = snap[snap["ind_gap_ma60"].isna() | (snap["ind_gap_ma60"] > 0)]   # 產業在
季線下的不買
        scores = bt.score_strategies(snap, p.esg_weight, risk_filters=p.risk_control)
        for strat, sc in scores.items():
            w = bt.make_weights(sc, p.top_n, p.weighting, industries=industries,
                                max_per_industry=config.MAX_PER_INDUSTRY)
            if d in schedule:
                targets[strat][d] = w
            hold_rows.append(pd.DataFrame({"date": d, "portfolio": strat, "ticker": w.index,
                                           "weight": w.values, "score":
sc.reindex(w.index).values}))
…（第 139 – 147 行略）
    ret_wide = full_panel.pivot_table(index="date", columns="ticker",
values="ret_1d").sort_index()
    risk = None
    if p.risk_control or p.pullback_control:
```

```
        if ds.benchmarks.empty or config.MARKET_INDEX not in set(ds.benchmarks["ticker"]):
            notes.append("找不到加權指數（^TWII），大盤指標改用股票池等權平均計算。")
        expo = bt.combine_exposure(
            bt.market_regime(ds.benchmarks, fallback=ret_wide) if p.risk_control else None,
            bt.pullback_exposure(ds.benchmarks, fallback=ret_wide) if p.pullback_control
else None)
        risk = bt.RiskControl(market_exposure=expo)
    series, trades, risk_rows = [], [], []
    for strat in bt.STRATEGIES:
        if not targets[strat]:
            why = "模型無預測" if strat == bt.STRATEGY_AI else "缺少 ESG 資料"
            notes.append(f"「{strat}」沒有可用的選股（{why}）。")
            continue
        sim = bt.simulate(targets[strat], ret_wide, risk)
        sim["portfolio"] = strat
        series.append(sim[["date", "portfolio", "ret", "nav"]])
        trades.append(sim.loc[sim["turnover"] > 0, ["date", "portfolio", "turnover",
"cost"]])
        risk_rows.append(sim[["date", "portfolio", "exposure"]])
```

程式 24　src/features.py：技術面與見頂訊號特徵、預測目標 add_technical()（第 78 – 116 行）

```
def add_technical(prices: pd.DataFrame, hold_days: int) -> pd.DataFrame:
    """技術面特徵＋風險控制用的欄位。只用當天（含）以前的資料。"""
    cols = ["date", "ticker", "close"] + (["volume"] if "volume" in prices.columns else [])
    df = prices[cols].copy()
    df["date"] = pd.to_datetime(df["date"]).astype("datetime64[ns]")
    df = df.dropna(subset=["date", "ticker", "close"]).sort_values(["ticker", "date"]) \
           .drop_duplicates(["ticker", "date"]).reset_index(drop=True)
    g = df.groupby("ticker", group_keys=False)["close"]
    df["ret_1d"] = g.pct_change()
    for n in (5, 20, 60):
        df[f"mom_{n}"] = g.pct_change(n)
    rg = df.groupby("ticker")["ret_1d"]
    df["vol_20"] = _roll(rg, 20, "std") * np.sqrt(config.TRADING_DAYS)
    ma20, ma60 = _roll(g, 20, "mean"), _roll(g, 60, "mean")
    df["bias_20"] = df["close"] / ma20 - 1
    df["gap_ma60"] = df["close"] / ma60 - 1
    # ---- 見頂訊號 ----
    hi20 = _roll(g, 20, "max", 10)
    df["dd_high20"] = df["close"] / hi20 - 1                        # 從近 20 日最高價回落多少（0
= 正在創高）
    df["days_since_high20"] = _days_since_high(df, 20, 10)          # 高點是幾天前（越大代表漲勢
停了越久）
    down = (df["ret_1d"] < 0).astype(int)
    grp = (down != down.groupby(df["ticker"]).shift()).cumsum()
    df["down_streak"] = down.groupby([df["ticker"], grp]).cumsum().where(down == 1, 0)   #
連續下跌天數
    if "volume" in df.columns:
        vg = df.groupby("ticker")["volume"]
        v60 = _roll(vg, 60, "mean", 40)
        df["vol_ratio"] = (_roll(vg, 5, "mean") / v60.where(v60 > 0))  # >1 放量、<1 量縮
    else:
        df["vol_ratio"] = np.nan
    df["reversal"] = df["mom_5"] * df["mom_60"]                     # 大漲後短線轉弱 → 負值
    # ---- 風險控制 ----
    # 個股長期趨勢：收盤價在年線（200 日均線）之上；上市不滿約 150 天無法判斷 → NaN（不排除）
    ma_long = _roll(g, config.STOCK_TREND_MA, "mean", int(config.STOCK_TREND_MA * 0.75))
    df["above_ma_long"] = (df["close"] > ma_long).astype(float).where(ma_long.notna())
    # 近 60 日年化波動（用來排除全體最震盪的 20%）
    df["vol_60"] = _roll(rg, 60, "std", 40) * np.sqrt(config.TRADING_DAYS)
    # 標籤：從今天收盤買進、持有 hold_days 個交易日的報酬（只用於訓練與評估）
    df["fwd_ret"] = g.shift(-hold_days) / df["close"] - 1
    return df
```

程式 25　src/features.py：依公告日對齊 ESG 與財務資料 asof_merge()（第 150 – 173 行）

```
def asof_merge(panel: pd.DataFrame, other: Optional[pd.DataFrame], cols: List[str],
               static: bool = False) -> pd.DataFrame:
    """把 other（ticker, available_date, cols...）依公布日對齊到 panel（date, ticker）。
    static=True：每檔一律用最新一筆（會有前視偏差，只在歷史資料不足時作為對照）。"""
    if other is None or other.empty or not cols:
        return panel
    o = other[["ticker", "available_date", *cols]].copy()
    o["available_date"] = pd.to_datetime(o["available_date"]).astype("datetime64[ns]")
```

```
    if static:
        latest = o.sort_values("available_date").groupby("ticker")[cols].last()
        return panel.merge(latest, left_on="ticker", right_index=True, how="left")
    o = o.dropna(subset=["available_date"]).sort_values("available_date")
    # 每一欄各自取「最新的非空值」，避免某欄這期沒填把舊值蓋掉
    left = panel.copy()
    left["date"] = pd.to_datetime(left["date"]).astype("datetime64[ns]")
    left = left.sort_values("date")
    for c in cols:
        oc = o.dropna(subset=[c])[["ticker", "available_date", c]]
        if oc.empty:
            left[c] = np.nan
            continue
        left = pd.merge_asof(left, oc, left_on="date", right_on="available_date",
by="ticker",
                             direction="backward").drop(columns=["available_date"])
    return left.sort_values(["ticker", "date"]).reset_index(drop=True)
```

程式 26　src/backtest.py：調倉日 rebalance_schedule()（第 38 – 53 行）

```
def rebalance_schedule(trading_days: pd.DatetimeIndex, first_allowed: pd.Timestamp,
                       every: int = None, mode: str = None) -> List[pd.Timestamp]:
    """調倉日。mode='calendar'（網站預設）：從 first_allowed 起每 2 週同一天，遇假日順延到下一個交易日；
    mode='trading_days'（論文）：每 every 個交易日一次。起點固定，資料變長時過去的調倉日不會移動。"""
    mode = mode or config.REBALANCE_MODE
    days = trading_days[trading_days >= first_allowed]
    if len(days) == 0:
        return []
    if mode == "trading_days":
        return [pd.Timestamp(d) for d in days[::max(1, every or config.HOLD_DAYS)]]
    out = []
    for a in rebalance_anchors(days[0], days[-1]):
        pos = days.searchsorted(a)
        if pos < len(days) and (not out or days[pos] > out[-1]):
            out.append(pd.Timestamp(days[pos]))
    return out
```

程式 27　src/models.py：滾動式訓練與預測 walk_forward()（第 101 – 144 行）

```
def walk_forward(panel: pd.DataFrame, features: List[str], rebalance_dates:
List[pd.Timestamp],
                 trading_days: pd.DatetimeIndex, hold_days: int, model_name: str,
n_estimators: int,
                 retrain_every: int = 1, required: Optional[List[str]] = None,
                 min_train_rows: int = 100, sample_every: int = 5, progress=None) ->
WalkForwardResult:
    """sample_every：訓練樣本每隔幾個交易日取一天。相鄰幾天的未來報酬高度重疊，
    全取只會變慢、不會多學到東西；每 5 天取一次速度快約 5 倍。"""
    required = required or []
    sampled_days = set(trading_days[::max(1, sample_every)])
    day_pos = {d: i for i, d in enumerate(trading_days)}
    preds, train_log = [], []
    model, used, medians, last_X = None, [], None, None

    for k, t in enumerate(rebalance_dates):
        if progress:
            progress(k, len(rebalance_dates), t)
        pos = day_pos[t]
        cutoff = trading_days[pos - hold_days] if pos - hold_days >= 0 else None
        if cutoff is None:
            continue
        if model is None or k % retrain_every == 0:
            train = panel[(panel["date"] <= cutoff) & panel["date"].isin(sampled_days)] \
                .dropna(subset=["fwd_ret", *required])
            cols = [c for c in features if c in train.columns and train[c].notna().any()]
            if len(train) < min_train_rows or not cols:
                continue
            medians = train[cols].median()
            X = train[cols].fillna(medians)
            y = train["fwd_ret"] - train.groupby("date")["fwd_ret"].transform("mean") if
config.TARGET_RELATIVE \
                else train["fwd_ret"]
            model = Model(model_name, n_estimators).fit(X, y)
            used = cols
```

```
            train_log.append({"rebalance_date": t, "train_end": cutoff, "n_samples": len(train),
                              "n_features": len(cols),
                              "importances": dict(zip(cols, map(float, model.feature_importances_)))})
        today = panel[panel["date"] == t].dropna(subset=required)
        if today.empty:
            continue
        X_t = today[used].fillna(medians)
        preds.append(pd.DataFrame({"date": t, "ticker": today["ticker"].values,
                                   "pred": model.predict(X_t)}))
        last_X = X_t

    pred_df = pd.concat(preds, ignore_index=True) if preds else pd.DataFrame(columns=["date", "ticker", "pred"])
    return WalkForwardResult(pred_df, model, used, last_X, train_log)
```

程式 28　src/backtest.py：三種策略的選股分數 apply_esg_rules()、score_strategies()（第 74 – 81、108 – 134 行）

```
def apply_esg_rules(df: pd.DataFrame) -> pd.DataFrame:
    """排除爭議分數 > 3 與 EPS 為負的公司（欄位存在時才套用）。"""
    out = df
    if "controversy_score" in out.columns:
        out = out[out["controversy_score"].isna() | (out["controversy_score"] <= 3)]
    if "eps" in out.columns:
        out = out[out["eps"].isna() | (out["eps"] >= 0)]
    return out
…（第 82 – 107 行略）
def score_strategies(snapshot: pd.DataFrame, esg_weight: float, risk_filters: bool = False,
                     industry_weight: Optional[float] = None) -> Dict[str, pd.Series]:
    """snapshot：某個調倉日所有股票（ticker, pred, esg_total...）。回傳各策略的分數（越高越好）。
    risk_filters=True：先排除跌破年線、波動最高 20% 的股票，再排序。
    industry_weight：AI 分數裡產業趨勢的比重（None = config.INDUSTRY_TREND_WEIGHT）。"""
    if industry_weight is None:
        industry_weight = config.INDUSTRY_TREND_WEIGHT
    snap = snapshot.set_index("ticker")
    if risk_filters:
        snap = apply_risk_filters(snap)
    scores: Dict[str, pd.Series] = {}
    if snap.empty:
        return scores
    if "pred" in snap.columns and snap["pred"].notna().any():
        scores[STRATEGY_AI] = ai_score(snap, industry_weight)
        filt = apply_esg_rules(snap.dropna(subset=["pred"]))
        ai_rank = _pct_rank(ai_score(filt, industry_weight))
        if "esg_total" in filt.columns and filt["esg_total"].notna().any():
            esg_rank = _pct_rank(filt["esg_total"]).fillna(0.5)   # 沒有評等 → 視為中間值
            scores[STRATEGY_AI_ESG] = (1 - esg_weight) * ai_rank + esg_weight * esg_rank
        else:
            scores[STRATEGY_AI_ESG] = ai_rank
    if "esg_total" in snap.columns:
        e = apply_esg_rules(snap)["esg_total"].dropna()
        if not e.empty:
            scores[STRATEGY_ESG] = e
    return scores
```

程式 29　src/backtest.py：每日淨值模擬 simulate()（第 208 – 250 行）

```
def simulate(targets: Dict[pd.Timestamp, pd.Series], ret_wide: pd.DataFrame,
             risk: Optional[RiskControl] = None) -> pd.DataFrame:
    """targets：{調倉日: 目標權重（總和 ≤ 1，其餘為現金）}；ret_wide：日報酬寬表（index=date）。
    回傳 date, ret, nav, turnover, cost, exposure（持股比例）。"""
    cols = ["date", "ret", "nav", "turnover", "cost", "exposure"]
    if not targets:
        return pd.DataFrame(columns=cols)
    days = ret_wide.index[ret_wide.index >= min(targets)]
    w = pd.Series(dtype=float)          # 目前持股權重；1 - 總和 = 現金
    base = pd.Series(dtype=float)       # 這個月的目標名單（滿倉時的權重）
    nav, scale = 1.0, 1.0
    rows = []
    for d in days:
        r_day = 0.0
        if not w.empty:
```

```
            r = ret_wide.loc[d].reindex(w.index).astype(float).fillna(0.0)   # 停牌或無報價當天視為 0
            gross = float((w * (1 + r)).sum() + (1 - w.sum()))
            r_day = gross - 1
            if gross > 0:
                w = w * (1 + r) / gross          # 權重隨價格漂移（買入持有）
        new = None
        if d in targets:
            base = targets[d]
            new = base * (risk.exposure(d) if risk else 1.0)
        elif risk is not None and not base.empty:
            e = risk.exposure(d)
            if abs(e - scale) >= risk.step - 1e-9:   # 只調整比例，不換股
                cur = w / w.sum() * base.sum() if w.sum() > 0 else base
                new = cur * e
        turnover = cost = 0.0
        if new is not None:
            new = new[new > 0]
            if risk is not None:
                scale = risk.exposure(d)
            idx = w.index.union(new.index)
            turnover = float((new.reindex(idx, fill_value=0) - w.reindex(idx, fill_value=0)).abs().sum() / 2)
            cost = trading_cost(w, new)
            w = new.copy()
        ret = (1 + r_day) * (1 - cost) - 1
        nav *= 1 + ret
        rows.append({"date": d, "ret": ret, "nav": nav, "turnover": turnover, "cost": cost,
                     "exposure": float(w.sum())})
    return pd.DataFrame(rows, columns=cols)
```

程式 30　src/backtest.py：短期轉弱減碼 pullback_exposure()、combine_exposure()（2026 年 11 月新增）（第 271 – 287、290 – 295 行）

```
def pullback_exposure(bench: pd.DataFrame, index: str = config.MARKET_INDEX, window: int = config.PULLBACK_WINDOW,
                      dd: float = config.PULLBACK_DD, weak: float = config.PULLBACK_EXPOSURE,
                      fallback: Optional[pd.DataFrame] = None) -> Optional[pd.Series]:
    """短期轉弱減碼：加權指數收盤價距近 window 日最高收盤價跌幅 ≤ dd → weak，否則 1.0。只用當天（含）以前的收盤價。"""
    px = None
    if bench is not None and not bench.empty and index in set(bench["ticker"]):
        px = bench[bench["ticker"] == index].set_index("date")["close"].sort_index()
        if fallback is not None:
            px = px.reindex(px.index.union(fallback.index)).ffill().reindex(fallback.index)
    elif fallback is not None and not fallback.empty:
        px = (1 + fallback.mean(axis=1).fillna(0)).cumprod()
    if px is None or px.notna().sum() < window:
        return None
    draw = px / px.rolling(window, min_periods=window).max() - 1
    out = pd.Series(np.where(draw <= dd, weak, 1.0), index=px.index)
    out[draw.isna()] = 1.0
    return out
…（第 288 – 289 行略）
def combine_exposure(*series: Optional[pd.Series]) -> Optional[pd.Series]:
    """多個減碼規則同時使用時取最低的持股比例。"""
    s = [x for x in series if x is not None]
    if not s:
        return None
    return pd.concat(s, axis=1).min(axis=1)
```

表 11　核心程式與論文研究方法對照

步驟	程式位置	論文對應
整體流程	src/pipeline.py 第 65 – 199 行（程式 23）	圖 1-1 研究流程
技術面特徵、見頂訊號、預測目標（未來 10 日報酬）	src/features.py 第 78 – 116 行（程式 24）	第三章第二節、表 3-5
ESG 與估值依公告日對齊	src/features.py 第 150 – 173 行（程式 25）	表 3-2 可得日規則

步驟	程式位置	論文對應
調倉日（論文與網站皆為每 2 週固定同一星期幾）	src/backtest.py 第 38–53 行（程式 26）	第三章第三節、表 3-9
Random Forest 滾動訓練（每 3 次調倉重訓、訓練樣本截至調倉日前 10 日）	src/models.py 第 101–144 行（程式 27）	第三章第四節（一）（二）
單純 AI、AI+ESG、規則式 ESG 篩選的分數	src/backtest.py 第 74–81、108–134 行（程式 28）	第三章第三、四節、表 3-6
前 N 檔與權重	src/backtest.py 第 148–164、167–176 行	Top 20 等權
每日淨值、週轉率與交易成本	src/backtest.py 第 180–186、208–250 行（程式 29、程式 15）	表 3-9 交易假設、表 4-12
績效指標與 CAPM	src/metrics.py 第 36–58、66–75 行（程式 10）	第三章第五節、表 4-1、表 4-4
短期轉弱減碼（選用，網站 2026 年 11 月新增）	src/backtest.py 第 271–287、290–295 行（程式 30）	論文未使用（表 3-1：不使用風險控制）
大盤狀態特徵（設定開關，預設關閉）	src/features.py 第 39–51 行	論文未使用；已測試、效果不一致

表 11 依執行順序列示選股與回測各步驟所在之程式位置及其對應之論文章節或表格，可據以由論文之方法說明追溯實際執行之程式；最後二列為資料截點後新增之選用功能，論文結果未使用。

十一、資料蒐集、自動更新與部署

網站資料由 GitHub Actions 於每個交易日台灣時間 14:50 自動更新：執行 update_data.py --auto 補抓到最近收盤日的股價、比較基準、三大法人、大盤長期資料與月底估值，再以 run_backtest.py 預先計算預設設定的回測，最後將資料與回測存檔提交回 GitHub，Streamlit Cloud 偵測到更新後自動重新部署。程式修改則由本機的 auto_push.py 監看：檔案變動後先執行測試，通過才提交並推送。

程式 31　src/store.py：每日自動更新 auto_update()（節錄）（第 275 – 313 行）

```
275 def auto_update(progress=None, flows_days: int = 10, now: Optional[datetime] = None) ->
    Dict:
276     """補到最近收盤日：股價（沿用現有股票池）、比較基準、最近幾天的三大法人、缺少的月底估值。
277     和 TEJ 完全無關；ESG 檔沒更新也照樣更新股價。任何一步失敗都不影響其他步驟。"""
278     target = last_close_date(now)
279     AUTO_STATE_FILE.parent.mkdir(parents=True, exist_ok=True)
280     AUTO_STATE_FILE.write_text(json.dumps({"target": str(target.date()),
281                                            "tried_at":
    str(pd.Timestamp(datetime.now(TW_TZ).replace(tzinfo=None)))}),
282                                encoding="utf-8")
283     existing = _read(config.PRICES_FILE)
284     tickers = sorted(existing["ticker"].unique()) if existing is not None else
    list(config.CORE_UNIVERSE)
285     hist_start = pd.Timestamp(config.HISTORY_START)
286     have_start = existing["date"].min() if existing is not None else None
287     # 歷史還沒補到 HISTORY_START（例如原本只有 3 年）→ 這次把所有股票往前補；補過之後就只往後補
288     backfill = have_start is None or have_start > hist_start + pd.Timedelta(days=30)
289     start = hist_start if backfill else have_start
290     out: Dict = {"target": str(target.date())}
291     report = (lambda msg: progress(msg)) if progress else (lambda msg: None)
292     if len(tickers) < config.UNIVERSE_SIZE * 0.9:
293         # 股票池比設定小很多（例如原本只有 50 檔）→ 自動擴充到成交金額前 N 大，新股票會補抓完整歷史
294         try:
295             report(f"股票池只有 {len(tickers)} 檔，擴充到成交金額前 {config.UNIVERSE_SIZE} 大")
296             listing = _read(config.LISTING_FILE, date_cols=())
297             top = top_universe(listing if listing is not None else
    pd.DataFrame(columns=["ticker"]))
298             tickers = sorted(set(tickers) | set(top))
299             out["universe"] = len(tickers)
300         except Exception as e:
301             out["universe"] = {"error": str(e)}
302     try:
303         report(f"股價：{len(tickers)} 檔補到 {target.date()}")
304         r = update_prices(tickers, start, target, backfill=backfill)
305         out["prices"] = {"rows": r["rows_downloaded"], "failed": len(r["failed"])}
306     except Exception as e:
307         out["prices"] = {"error": str(e)}
308     try:
309         report("比較基準（加權指數、ETF）")
310         b = update_benchmarks(start, target, backfill=backfill)
311         out["benchmarks"] = {"rows": b["rows_downloaded"]}
312     except Exception as e:
313         out["benchmarks"] = {"error": str(e)}
```

程式 32　src/crawlers/yahoo.py：Yahoo Finance 批次下載 fetch_prices()（節錄）（第 60 – 87 行）

```
 60 def fetch_prices(tickers: List[str], start, end, chunk: int = 50,
 61                  progress: Optional[Callable[[int, int, str], None]] = None) ->
    Tuple[pd.DataFrame, List[str]]:
 62     """批次下載還原權息後的日 K。回傳 (長表, 失敗代號)。
 63     progress(完成數, 總數, 階段說明)。沒抓到的代號只再「批次」重試一輪，不再一檔一檔重試
 64     （舊做法遇到上百檔抓不到時，會每檔等 2 秒、卡住十幾分鐘）。"""
 65     if not YF_AVAILABLE:
 66         raise ImportError("尚未安裝 yfinance：pip install yfinance")
 67     tickers = list(dict.fromkeys(t for t in tickers if t))
 68     if not tickers:
```

```
        return pd.DataFrame(columns=PRICE_COLS), []
    end_excl = (pd.Timestamp(end) + pd.Timedelta(days=1)).strftime("%Y-%m-%d")  # yfinance 的 end 不含當天
    start = pd.Timestamp(start).strftime("%Y-%m-%d")

    def _download(batch):
        raw = yf.download(batch, start=start, end=end_excl, auto_adjust=True, group_by="ticker",
                          threads=True, progress=False)
        return parse_yf_frame(raw, batch)

    got = []
    for i in range(0, len(tickers), chunk):
        batch = tickers[i:i + chunk]
        try:
            got.append(_download(batch))
        except Exception as e:
            log.warning("批次下載失敗（%s...）：%s", batch[0], e)
        if progress:
            progress(min(i + chunk, len(tickers)), len(tickers), "下載股價")
        time.sleep(1 + random.random())
```

程式 33　.github/workflows/daily_update.yml：每日自動更新排程（GitHub Actions）（第 1 – 49 行）

```
# 每天自動抓最新股價並更新網站（在 GitHub 的主機上執行，你的電腦不用開機）
# 時間：週一到週五 台灣時間 14:50（UTC 06:50）。也可以在 GitHub 網頁的 Actions 分頁按「Run workflow」手動執行。
name: 每日更新股價

on:
  schedule:
    - cron: "50 6 * * 1-5"
  workflow_dispatch:

permissions:
  contents: write          # 允許把更新後的資料 commit 回這個 repo

concurrency:
  group: daily-update
  cancel-in-progress: false

jobs:
  update:
    runs-on: ubuntu-latest
    timeout-minutes: 30
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip

      - name: 安裝套件
        run: pip install -r requirements.txt

      - name: 抓最新股價（Yahoo）與比較基準；證交所資料抓不到會略過，不影響股價
        run: python scripts/update_data.py --auto

      - name: 重算預設設定的回測（網站打開就有結果）
        run: python scripts/run_backtest.py

      - name: 有新資料就 commit 回 repo（Streamlit Cloud 會自動重新部署）
        run: |
          git config user.name "github-actions[bot]"
          git config user.email "41898282+github-actions[bot]@users.noreply.github.com"
          git add data/processed outputs/cache
          if git diff --cached --quiet; then
            echo "沒有新資料"
          else
            git commit -m "每日更新股價 $(date -u +%F)"
            git pull --rebase --autostash
            git push
          fi
```

程式 34　scripts/auto_push.py：程式修改後自動測試與推送 publish()（第 100 – 103、114 – 141 行）

```
100 def publish(files: list[str]) -> bool:
101     code = [f for f in files if f.endswith(".py")]
102     if code and not run_tests():
103         return False
    …（第 104 – 113 行略）
114     pull = git("pull", "--rebase", "--autostash")
115     if pull.returncode != 0:
116         git("rebase", "--abort")
117         log(f"  × 無法和 GitHub 上的版本合併，請用 GitHub Desktop 處理：{(pull.stderr or
    pull.stdout).strip()[-600:]}")
118         return False
119     if CLOUD_DATA:
120         git("add", "-A", "--", ".", *[f":(exclude){p}" for p in DATA_PATHS], check=True)
121     else:
122         git("add", "-A", check=True)
123     if not git("diff", "--cached", "--name-only").stdout.strip():
124         return True
125     names = [Path(f).name for f in files]
126     summary = "、".join(names[:5]) + (f" 等 {len(names)} 個檔案" if len(names) > 5 else "")
127     git("commit", "-m", f"auto: 更新 {summary}", check=True)
128     r = git("push")
129     if r.returncode != 0:
130         log("  push 被拒，先同步 GitHub 上的新版本再推一次 …")
131         pull = git("pull", "--rebase", "--autostash")
132         if pull.returncode != 0:
133             git("rebase", "--abort")
134             log(f"  × 無法自動合併，請用 GitHub Desktop 處理：{(pull.stderr or
    pull.stdout).strip()[-600:]}")
135             return False
136         r = git("push")
137     if r.returncode != 0:
138         log(f"  × push 失敗：{(r.stderr or r.stdout).strip()[:300]}")
139         return False
140     log(f"  √ 已推到 GitHub（{summary}），網站約 1～3 分鐘後自動更新")
141     return True
```

表 12　資料來源與更新程式對照

資料	來源	程式位置	更新方式
個股股價（網站）	Yahoo Finance	src/crawlers/yahoo.py 第 60 – 108 行（程式 32）	每日增量補抓；失敗的股票批次重試
比較基準（加權指數、0050、00850、00878）	Yahoo Finance	src/store.py 第 137 – 138 行	每日
上市櫃清單與產業	證交所 ISIN	src/crawlers/twse.py 第 90 – 99 行	手動更新時
本益比、淨值比、殖利率	證交所 BWIBBU	src/crawlers/twse.py 第 136 – 156 行	每月月底一筆
三大法人買賣超	證交所 T86	src/crawlers/twse.py 第 198 – 223 行	每日最近 10 個交易日
大盤長期資料（加權指數、S&P 500、VIX 等）	Yahoo Finance	src/store.py 第 338 – 353 行	每日
TEJ TESG 評等	TEJ 下載檔	src/loaders/tej.py 第 70 – 87 行	新一期公告後放入 data/raw/tej/
每日排程	GitHub Actions	.github/workflows/daily_update.yml（程式 33）	週一至週五 14:50
自動更新主程式	—	src/store.py 第 275 – 335 行（程式 31）	任一步驟失敗不影響其他步驟
程式部署	GitHub → Streamlit Cloud	scripts/auto_push.py 第 100 – 141 行（程式 34）	測試通過才推送

表 12 列示網站使用之各項資料、資料來源、下載程式位置與更新頻率，據以區分每日自動更新與需手動更新之資料。

十二、論文研究程式與輸出檔

論文的正式結果由 run_research.py 依 research_config.json 的固定設定產生：先以逐日行情建立含下市股票的價格面板與逐期股票池，再呼叫第十節的 pipeline.run() 回測，最後計算各項統計檢定並輸出表格檔與 manifest.json（設定、資料與程式的 SHA-256）。build_report.py 讀取這些輸出檔產生論文，文中的數字全部由輸出檔讀取。

程式 35　scripts/research/run_research.py：研究資料集與逐期股票池 build_dataset()（第 77 – 122 行）

```
 77  def build_dataset(cfg: dict, out: Path, test_yahoo: bool = False):
 78      cut = pd.Timestamp(cfg["data_cutoff"])
 79      t0 = time.time()
 80      if test_yahoo:
 81          q, px = yahoo_as_quotes(load_dataset("real"), cut)
 82      else:
 83          q = ex.load_quotes(end=cut)
 84          days_tp = q.loc[q["market"] == "TPEx", "date"].nunique()
 85          days_tw = q.loc[q["market"] == "TWSE", "date"].nunique()
 86          fm = ex.load_finmind_twse(end=cut)
 87          if days_tw >= 0.98 * days_tp:          # 證交所逐日全市場行情齊全
 88              px = ex.build_prices(q)
 89              source = "證交所 MI_INDEX＋櫃買中心 dailyQuotes（逐日全市場）"
 90          else:                                   # 上市：Yahoo（現存）＋ FinMind（已下市）；上櫃：櫃買中心逐日行情
 91              ya = ex.load_yahoo_all(end=cut)
 92              px = ex.build_prices_hybrid(q, fm, ya)
 93              n_fm = len(set(fm["ticker"]) - set(ya["ticker"])) if len(fm) else 0
 94              source = (f"上市：Yahoo Finance 逐檔日資料（{ya['ticker'].nunique()} 檔現存股票）＋FinMind 逐檔行情（{n_fm} 檔 Yahoo 未收錄、多為已下市）；"
 95                        f"上櫃：櫃買中心 dailyQuotes 逐日全市場行情")
 96          base0 = load_dataset("real")
 97          px = ex.apply_yahoo_returns(px, base0.prices[base0.prices["date"] <= cut])
 98          share = (px["ret_source"] == "Yahoo").mean()
 99          source += f"；現存 300 檔的日報酬改用 Yahoo 還原收盤價（占全部股票日資料 {share:.0%}），其餘股票（含已下市）用漲跌價差法"
100          print("價格來源：", source, flush=True)
101          (out / "price_source.txt").write_text(source, encoding="utf-8")
102      u = cfg["universe"]
103      elig = ex.eligibility(px, u["size"], u["window_days"], u["min_trading_days"])
104      print(f"行情 {px['date'].nunique()} 天、{px['ticker'].nunique()} 檔；股票池建立 {time.time()-t0:.0f}s", flush=True)
105      ever = set(elig["ticker"])
106      base = load_dataset("real")
107      names = q.drop_duplicates("ticker", keep="last").set_index("ticker")["name"]
108      listing = base.listing.copy()
109      extra = pd.DataFrame({"ticker": sorted(ever - set(listing["ticker"]))})
110      if len(extra):
111          extra["code"] = extra["ticker"].str[:4]
112          extra["name"] = extra["ticker"].map(names)
113          extra["market"] = np.where(extra["ticker"].str.endswith(".TWO"), "上櫃", "上市")
114          extra["industry"] = None
115          listing = pd.concat([listing, extra], ignore_index=True)
116      bench = base.benchmarks[base.benchmarks["date"] <= cut]
117      esg = base.esg[base.esg["available_date"] <= cut]
118      fin = base.financials[base.financials["available_date"] <= cut]
119      prices = px[px["ticker"].isin(ever)][["date", "ticker", "close", "volume"]].copy()
120      ds = Dataset(mode="research", prices=prices, benchmarks=bench, esg=esg, financials=fin,
121                   listing=listing, notes=[], eligible=elig[["date", "ticker"]])
122      return ds, px, elig, base
```

程式 36　src/research/exchange.py：逐期股票池（調倉日以前 20 日平均成交金額前 300 大）eligibility()（第 199 – 213 行）

```
199 def eligibility(prices: pd.DataFrame, size: int = 300, window: int = 20, min_days: int = 15)
    -> pd.DataFrame:
200     """每個交易日 t：以 t 當天（含）往前 window 個交易日的平均成交金額排名，取前 size 檔。
201     交易天數不足 min_days（新上市）不納入。回傳 date,ticker,avg_value,n_days,rank（只含入選者）。"""
202     days = pd.DatetimeIndex(sorted(prices["date"].unique()))
203     v = prices.pivot_table(index="date", columns="ticker", values="value",
    aggfunc="sum").reindex(days)
204     listed = v.notna()
205     avg = v.fillna(0).rolling(window, min_periods=1).sum() / window
206     n = listed.rolling(window, min_periods=1).sum()
207     avg = avg.where(n >= min_days)
208     rk = avg.rank(axis=1, ascending=False, method="first")
209     long = pd.DataFrame({"avg_value": avg.stack(), "n_days": n.stack(), "rank":
    rk.stack()}).dropna().reset_index()
210     long.columns = ["date", "ticker", "avg_value", "n_days", "rank"]
211     out = long[long["rank"] <= size].copy()
212     out["rank"] = out["rank"].astype(int)
213     return out
```

程式 37　src/research/stats.py：統計檢定：HAC 平均數、CAPM、差異檢定、Holm 調整、區塊 bootstrap（第 20 – 29、32 – 46、49 – 56、59 – 69、72 – 83 行）

```
 20 def hac_mean(x: pd.Series) -> Dict:
 21     """平均數的 HAC 檢定（對常數迴歸）。回傳估計、HAC 標準誤、t、p、95% 信賴區間、樣本數、落後期數。"""
 22     x = pd.Series(x).dropna().astype(float)
 23     n = len(x)
 24     L = nw_lags(n)
 25     m = sm.OLS(x.values, np.ones(n)).fit(cov_type="HAC", cov_kwds={"maxlags": L})
 26     est, se = float(m.params[0]), float(m.bse[0])
 27     ci = m.conf_int(0.05)[0]
 28     return {"n": n, "lags": L, "est": est, "se": se, "t": est / se if se else np.nan,
 29             "p": float(m.pvalues[0]), "ci_lo": float(ci[0]), "ci_hi": float(ci[1])}
…（第 30 – 31 行略）
 32 def capm(rp: pd.Series, rm: pd.Series, rf_annual: float = 0.01) -> Dict:
 33     """日頻 CAPM：(rp − rf) = α + β (rm − rf) + ε，HAC 標準誤。"""
 34     df = pd.concat([rp, rm], axis=1, keys=["p", "m"]).dropna()
 35     rf = (1 + rf_annual) ** (1 / TD) - 1
 36     y, x = df["p"] - rf, df["m"] - rf
 37     n = len(df)
 38     L = nw_lags(n)
 39     m = sm.OLS(y.values, sm.add_constant(x.values)).fit(cov_type="HAC", cov_kwds={"maxlags":
    L})
 40     a, b = m.params
 41     sa, sb = m.bse
 42     ci = m.conf_int(0.05)
 43     return {"n": n, "lags": L, "alpha_d": float(a), "alpha_se": float(sa), "alpha_t":
    float(a / sa),
 44             "alpha_p": float(m.pvalues[0]), "alpha_ci_lo": float(ci[0][0]), "alpha_ci_hi":
    float(ci[0][1]),
 45             "alpha_ann": float((1 + a) ** TD - 1), "beta": float(b), "beta_se": float(sb),
 46             "beta_ci_lo": float(ci[1][0]), "beta_ci_hi": float(ci[1][1]), "r2":
    float(m.rsquared)}
…（第 47 – 48 行略）
 49 def diff_test(rp: pd.Series, rb: pd.Series) -> Dict:
 50     """策略與比較基準的日報酬差：HAC 平均差檢定＋效果量（年化差、日差 ÷ 日差標準差）。"""
 51     d = (pd.concat([rp, rb], axis=1, keys=["p", "b"]).dropna().pipe(lambda x: x["p"] -
    x["b"]))
 52     out = hac_mean(d)
 53     out["ann_diff"] = out["est"] * TD
 54     out["effect_d"] = float(d.mean() / d.std(ddof=1)) if d.std(ddof=1) > 0 else np.nan
 55     out["ir_ann"] = out["effect_d"] * math.sqrt(TD)      # 年化資訊比率
 56     return out
…（第 57 – 58 行略）
 59 def holm(pvals: List[float]) -> List[float]:
 60     """Holm-Bonferroni 調整後 p 值（控制整體型一錯誤率）。"""
 61     p = np.asarray(pvals, float)
 62     order = np.argsort(p)
 63     m = len(p)
 64     adj = np.empty(m)
 65     run = 0.0
 66     for k, i in enumerate(order):
 67         run = max(run, min(1.0, (m - k) * p[i]))
 68         adj[i] = run
```

```
    return adj.tolist()
…（第 70–71 行略）
def bootstrap_ci_ann(r: pd.Series, block: int = 20, n_boot: int = 2000, seed: int = 42) -> tuple:
    """移動區塊 bootstrap 的年化幾何報酬 95% 信賴區間（保留短期自我相關）。"""
    x = pd.Series(r).dropna().values
    n = len(x)
    rng = np.random.default_rng(seed)
    nb = int(np.ceil(n / block))
    out = []
    for _ in range(n_boot):
        starts = rng.integers(0, n - block + 1, nb)
        s = np.concatenate([x[i:i + block] for i in starts])[:n]
        out.append(np.prod(1 + s) ** (TD / n) - 1)
    return float(np.percentile(out, 2.5)), float(np.percentile(out, 97.5))
```

程式 38　scripts/research/run_research.py：表 4-1 績效與信賴區間的計算（第 289–303 行）

```
    # 表 4-1 績效＋CAPM（HAC）＋報酬 bootstrap CI
    t41 = []
    for c in W.columns:
        r = W[c].dropna()
        from src.metrics import performance
        m = performance(r, config.RISK_FREE_RATE)
        lo, hi = rs.bootstrap_ci_ann(r)
        row = {"投組": c, "累積報酬": m["cum_return"], "年化報酬": m["ann_return"], "年化報酬95%CI下限": lo,
               "年化報酬95%CI上限": hi, "年化波動": m["ann_vol"], "Sharpe": m["sharpe"], "最大回撤": m["max_drawdown"],
               "交易日數": len(r)}
        if c != "加權指數":
            cp = rs.capm(r, mkt, config.RISK_FREE_RATE)
            row.update({"Beta": cp["beta"], "年化Alpha(CAPM)": cp["alpha_ann"], "R2": cp["r2"]})
        t41.append(row)
    pd.DataFrame(t41).to_csv(tdir / "table_4_1_performance.csv", index=False)
```

註：run_research.py 輸出檔名與程式註解中的表號沿用初版編號，與論文表號的對應見表 13。

表 13　論文表格、輸出檔與產生程式對照

論文表／圖	輸出檔（outputs/research/thesis_2026-10-20/）	產生程式
表 3-3 逐期股票池概況	universe_summary.csv、universe_history.csv、universe_screening.csv	src/research/exchange.py 第 199–213 行（程式 36）；exchange.universe_summary()
表 3-4 樣本期間切分	tables/table_3_7_period_split.csv	scripts/research/run_research.py main()
表 3-8 參數試驗紀錄	tables/table_3_8_parameter_trials.csv	parameter_trials.csv（探索階段紀錄）
表 4-1 績效比較	tables/table_4_1_performance.csv	scripts/research/run_research.py 第 289–303 行（程式 38）
表 4-2 各年度報酬	tables/table_4_2_yearly.csv	src/research/analysis.py yearly()
表 4-3 投組 ESG 品質；圖 4-2	tables/table_4_10_esg_quality.csv、esg_quality_by_rebalance.csv	src/research/analysis.py esg_quality()
表 4-4 風險指標比較	tables/table_4_1_performance.csv、table_4_4_capture.csv	build_report.py risk_table()
表 4-5 市場情境	tables/table_4_3_regimes.csv、regimes.csv	src/research/analysis.py classify_regimes()、regime_table()
表 4-6 上漲日／下跌日參與率	tables/table_4_4_capture.csv	src/research/analysis.py capture()

論文表／圖	輸出檔（outputs/research/thesis_2026-10-20/）	產生程式
表 4-7 CAPM 估計	tables/table_4_5_capm_hac.csv	src/research/stats.py capm()、holm()（程式 37）
表 4-8 日報酬差異檢定	tables/table_4_6_diff_hac.csv	src/research/stats.py diff_test()、holm()（程式 37）
表 4-9 模型選股能力	tables/table_4_7_model_skill.csv、rank_ic.csv	pipeline.rank_ic()、stats.hac_mean()
表 4-10 假說檢驗彙整	（由表 4-1、4-7、4-8 彙整）	build_report.py ch4()
表 4-11 股票池與調倉日期對照	tables/ table_4_2b_*.csv、table_4_11c_*.csv、strategy_returns_*.csv	scripts/research/run_research.py main()（--test-yahoo 等對照）
表 4-12 換股與交易成本	tables/table_4_9a_turnover_cost.csv	scripts/research/run_research.py main()
表 4-13 滑價敏感度	tables/table_4_9b_slippage.csv	scripts/research/run_research.py resimulate()
表 4-14 特徵重要性	tables/ table_4_8a_importance_distribution.csv、table_4_8b_permutation_importance.csv	scripts/research/run_research.py main()
圖 4-1 累積淨值	strategy_returns.csv	pipeline.run()；build_report.py figures()
附錄 A 版本紀錄	manifest.json	scripts/research/run_research.py main()

表 13 將論文之各表與各圖對應至其輸出檔與產生程式；檢查論文數據時，可據此追溯原始輸出檔與計算程式。

十三、程式測試

程式以 pytest 撰寫自動化測試，auto_push.py 在每次推送前執行；測試重點是確認回測沒有使用未來資訊、交易成本與報酬計算正確，以及各資料來源的解析格式。另以 smoke_ui.py 模擬 Streamlit 把每一頁從頭執行一次，確認畫面程式不會因資料處理錯誤而中斷。

表 14　主要測試項目（共 40 項，節錄）

測試函式	程式位置	檢查內容
test_asof_merge_uses_only_published_data	tests/test_core.py 第 177–184 行	ESG／財務資料只使用公告日以前已公告者
test_future_prices_do_not_change_past_decisions	tests/test_core.py 第 191–205 行	加入未來股價不會改變過去的選股
test_training_window_ends_before_rebalance	tests/test_core.py 第 208–214 行	訓練樣本截止於調倉日前 10 個交易日
test_simulate_returns_start_next_day_and_costs	tests/test_core.py 第 218–226 行	報酬自調倉日下一個交易日起算並扣成本
test_trading_cost_sell_side_has_tax	tests/test_core.py 第 229–232 行	賣出端含證交稅
test_strategies_share_same_period	tests/test_core.py 第 244–247 行	三種策略的比較期間一致
test_metrics_known_values	tests/test_core.py 第 37–44 行	績效指標與已知數值相符
test_capm_alpha_beta	tests/test_core.py 第 384–396 行	CAPM Beta 與詹森 Alpha
test_holding_forecast	tests/test_core.py 第 431–455 行	持有期間預估路徑
test_pullback_exposure_threshold_and_no_lookahead	tests/test_core.py 第 321–333 行	短期轉弱減碼的門檻正確、只用當天以前的收盤價
test_market_view_uses_only_past_and_follows_state	tests/test_core.py 第 458–480 行	大盤狀態預估只用已實現的資料；策略 Beta 計算正確
test_eligibility_top_n_uses_past_window_only	tests/test_research.py 第 34–44 行	逐期股票池只用過去 20 日成交金額
test_daily_returns_twse_and_tpex	tests/test_research.py 第 8–20 行	上市櫃日報酬（含權息）計算
test_hac_and_holm	tests/test_research.py 第 47–53 行	HAC 標準誤與 Holm 調整
test_regimes_use_previous_day	tests/test_research.py 第 64–69 行	市場情境只用前一日資訊

表 14 節錄主要之自動化測試及其檢查內容，重點在於確認回測未使用未來資料，以及交易成本與績效計算之正確性；每次推送程式至 GitHub 前均會自動執行全部測試。

程式 39　tests/test_core.py：前視偏差測試（範例）（第 191 – 205 行）

```
191  def test_future_prices_do_not_change_past_decisions():
192      """把某天之後的價格全部改掉，在那之前的選股必須完全不變。"""
193      ds = _demo()
194      p = Params(top_n=3, n_estimators=30)
195      base = run(ds, p)
```

```
    cut = base.rebalance_dates[2]
    shocked = ds.prices.copy()
    after = shocked["date"] > cut
    rng = np.random.default_rng(0)
    shocked.loc[after, "close"] *= rng.uniform(0.5, 1.5, after.sum())
    ds2 = store.Dataset(ds.mode, shocked, ds.benchmarks, ds.esg, ds.financials, ds.listing, [])
    alt = run(ds2, p)
    h1 = base.holdings[base.holdings["date"] <= cut].reset_index(drop=True)
    h2 = alt.holdings[alt.holdings["date"] <= cut].reset_index(drop=True)
    pd.testing.assert_frame_equal(h1, h2)
```